

Using the Schur Product to Solve the Code Equivalence Problem

Michele Battagliola¹, Rocco Mora², and Paolo Santini³

^{1,3} Università Politecnica delle Marche, Ancona, Italia

`battagliola.michele@proton.me`,

`p.santini@univpm.it`,

² CISP-Helmholtz Center for Information Security, Sankt Ingbert, Germany

`rocco.mora@cispa.de`

Abstract. Given two linear codes, the Code Equivalence Problem asks to find (if it exists) an isometry between them. A special case is the Permutation Equivalence Problem (PEP), where the isometry is a permutation. The hardness of PEP is crucially dependent on the *hull* of a code, that is, the intersection between a code and its dual. For random codes, with large probability the hull is trivial, i.e., has dimension $h = 0$: this allows for efficient attacks. However, when the hull is large enough, all known attacks take exponential time and PEP is deemed hard.

In this paper we study how the so-called Schur product between linear codes can be employed to solve PEP. The main idea is to transform a given PEP instance by computing powers of the given codes. We show that, while squaring a pair of equivalent codes preserves the equivalence, the new pair of codes have trivial hull with high probability. This allows to identify many new weak instances of PEP, namely: whenever $h < \sqrt{2n}$. With some technical caveats, our solver runs in average polynomial time. As a concrete application, we consider the updatable encryption scheme proposed by Albrecht, Benčina and Lai at Eurocrypt 2025. All the recommended instances fall into the range of weak PEP instances we identify in this paper, hence are susceptible to our attack. We successfully recover the secret permutation for one of the instances claiming 128 bits of security. As a fix, instances with hull dimension $h > \sqrt{2n}$ shall be employed.

1 Introduction

Given two linear codes, the code equivalence problem (in its decisional version) asks to decide whether there exists an *isometry* mapping one code into the other. The isometry can be seen as an isomorphism that preserves the weight distribution; from a geometric perspective, the two codes represent the same object and, de facto, the isometry can be thought of as a simple relabeling of the codewords. As for many other coding theory concepts and notions, the problem has traditionally been studied in the context of the *Hamming metric*. Moreover, normally one looks only at the case in which the considered isometry is *linear*:

the most general type of linear isometries corresponds to the group of monomial transformations, that is, permutations that also scale coordinates. In this case, the problem goes by the name of Linear Equivalence Problem (LEP). As special cases of LEP we can consider the Permutation Equivalence Problem (PEP), where all the coefficients of the monomial are 1, and the Signed Permutation Equivalence Problem (SPEP), where the coefficients of the monomial are either 1 or -1 .¹

Remarkably, the code equivalence problem has also been studied in other contexts, e.g., the case of rank metric and matrix codes [CNP⁺23,RST24,CDAG20]. Interestingly, PEP can also be seen as a special case of the Lattice Isomorphism Problem (LIP) for modular lattices (permutation matrices are a special case of orthonormal transformations) [DvW22]. In a paper presented at Eurocrypt 2025, by Albrecht, Benčina and Lai have used PEP as the foundational security assumption for an updatable encryption scheme based on lattices [ABL25].

Hardness of PEP In 1997, Petrank and Roth showed that PEP is unlikely to be NP-complete, unless the polynomial hierarchy collapses [PR97]. Moreover, in [SS13] Sendrier and Simos proposed a reduction which maps any LEP instance to a PEP instance. The reduction takes time which is polynomial in the finite field size: this implies that LEP is also unlikely to be NP-complete.²

It is immediate to see that the search version of PEP is at least as hard as the decisional version. Recently, Biasse and Micheli proposed a polynomial time reduction for the other way, i.e., from search PEP to decisional PEP [BM23].

In [PR97], the authors showed that Graph Isomorphism Problem (GIP) reduces to PEP. GIP has been believed hard for decades, until the recent breakthrough algorithm proposed by Babai [Bab16], showing that GIP can be solved in quasi-polynomial time. This algorithm makes the existence of efficient solvers for the code equivalence more likely. However, when it comes to practical attacks, the situation for the code equivalence problem is still uncertain.

Solvers for PEP The first algorithm to solve the code equivalence was proposed by Leon in 1982 [Leo82]; this algorithm was later improved by Beullens in 2020 [Beu20]. Even though these algorithms work for any pair of input codes, they require to first find low weight codewords, which is a notoriously hard problem in coding theory. So, they generically take time which is exponential in the code length. Another generic solver is based on finding collisions between canonical

¹ There exists also another definition of equivalence, called semi-linear LEP (Semi-LEP), in which one allows also for the application of a field automorphism.

² The same result also holds for Semi-LEP, since any instance can be transformed to a LEP instance by brute-forcing over the automorphisms of the finite field. All automorphisms of a finite field correspond to Frobenious automorphisms: unless the extension degree of the finite field is exponentially large, Semi-LEP reduces to LEP in polynomial time.

forms [CPS25,Now25]; using a meet-in-the-middle procedure, this takes time $O\left(\sqrt{\binom{n}{k}}\right)$, with n and k being the code length and dimension, respectively.

Another well-known solver for PEP is the Support Splitting Algorithm (SSA) [Sen00], proposed by Sendrier in 2000, which reconstructs the secret permutation by guessing its action on each index. SSA is based on enumerating the hull and has running time which, heuristically, can be estimated as $O(n^3 + n \log(n)q^h)$, with q being the finite field size and h the hull dimension. In 2019, Bardet et al. proposed a solver for the decisional version of PEP [BOST19]. They show that PEP for codes with trivial hull (i.e., with dimension 0) can be transformed, in polynomial time, into a GIP instance in which the input graphs are undirected and weighted (the edges are labeled with finite field elements). Thanks to Babai's algorithm [Bab16], we know that GIP can be solved in quasi-polynomial time, so that all of these PEP instances can be solved in quasi-polynomial time as well in the worst case. In [BOST19], the authors show also how to perform the reduction to GIP when hulls are non-trivial; in this case, the reduction runs in worst case time $O(n^{h+\omega})$, where ω is the exponent for matrix inversion.

In light of this discussion, we have that PEP can generally be solved easily whenever the hull dimension h is small. Since for random codes the hull is very small with high probability [Sen97], we have that both SSA and the reductions to GIP efficiently solve random instances of PEP. However, for all codes having a sufficiently large hull, these algorithms get slower and become inefficient. Since the other known algorithms require finding low weight codeword, instances with large hulls are deemed as hard.

The square code The square code is a coding theory construction that extends to codes the component-wise (Schur) product of two vectors. In the last 15 years, it turned out to be a powerful tool in the cryptanalysis of code-based cryptosystems, namely within the context of the McEliece scheme [McE78]. Here, security hinges, among other things, on the difficulty of distinguishing the public code from a random one or recovering the hidden structure.

For several McEliece variants, the square code has proven instrumental in structural attacks. In particular, these attacks exploit the fact that the square code of certain families of algebraic and structured codes has an unusually small dimension compared to that of a random linear code. Thus, by computing the square code of a public key, an attacker can detect nonrandom behaviors and obtain a distinguishing attack. Sometimes, this is the first step towards a recovery attack, as the knowledge of the low-dimension square code may lead to partial or full reconstruction of the secret key. A non-exhaustive list of codes suffering from these techniques is given by generalized Reed-Solomon (GRS) codes and their low-codimensional subcodes [Wie10], modified versions of GRS codes [CGGU⁺14], Reed-Muller codes [BC14], Wild Goppa codes [COT16], algebraic geometry codes [CMCP17], polar codes [DBB18], subspace subcodes of RS codes [CL21], high-rate alternant/Goppa codes [FGUO⁺13,MT23,BMT23], twisted GRS codes [CPTZ25].

A necessary condition for such a distinguisher to work is that the square code must not fill the ambient space, and this may impose conditions on the initial dimension of the target code. For instance, the binary Goppa codes used in the original proposal by McEliece or in the IND-CCA2 NIST candidate Classic McEliece [ABC⁺22] resist this kind of attack, as their dimension is big enough to provide a trivial square code.

We will show that a similar constraint applies to our work, which instead focuses on the use of the square code for solving the code equivalence problem.

1.1 Our contribution

In this paper we provide new insights and solvers for the code equivalence problem, finding several new weak instances for PEP.

In particular, we study how the Schur product can be employed as a rather handy tool to improve known solvers on the code equivalence problem. The underlying idea is actually very simple: instead of solving PEP on a pair of codes $(\mathcal{A}, \mathcal{B})$, we instead consider their powers $(\mathcal{A}^i, \mathcal{B}^i)$ for some integer $i \geq 2$. If $\mathcal{B} = \pi(\mathcal{A})$ for some permutation π , then one has also $\mathcal{B}^i = \pi(\mathcal{A}^i)$. Notice that the Schur product operation modifies heavily the code structure: for instance, it increases significantly the code dimension. For this reason, our approach can be employed only on codes having a sufficiently small dimension.

However, computing powers of codes affects the hull: even if $\mathcal{A}$ and $\mathcal{B}$ have large hull, this may not be true for $\mathcal{A}^i$ and $\mathcal{B}^i$. Even when considering codes which are not self-orthogonal, our attack can be applied by focusing only on the hulls.

In this paper, we focus only on the case $i = 2$, i.e., on the so-called *squaring operation*. We show that the hull dimension of the square of a self-orthogonal code has a somewhat different distribution with respect to that of random codes but, in any case, the hull is small with large probability. This implies that, with large probability, the PEP instance given by the square codes can be solved efficiently.

False positives and spurious solutions This simple approach actually hides some slight obstacles that need to be addressed. First, looking at the squared hull may lead to *false positives*. Indeed, it may happen that the squared hulls are permutation equivalent even if the initial codes are not. Analogously, when looking at the search version of PEP, one may find spurious solutions, i.e., permutation that solve PEP on the squared hulls but not on the initial codes. The existence of false positives and spurious solutions are somewhat intrinsic to the fact that the square of a self-orthogonal code has some special properties. For instance, regardless of the dimension, the dual of the square code always contains the all-ones codeword. This means that the squared hull will never be a degenerate code but, if the hull dimension is too large, its square always corresponds to the dual of $\text{Span}((1, \dots, 1))$. For such a reason, looking at the squared hulls is not expected to work when $\frac{h(h+1)}{2} = n - 1$.

More generally, whenever the squared hull has a dimension which is too large, say, $n - u$ for small u , false positives and spurious solutions are more likely to happen. Heuristically, we show that when $u > 1 + c \log_2(n)$ for some constant $c > 0$ that depends also on the finite field size, the impact of false positives becomes less meaningful. This condition shall be interpreted as a rule of thumb as we obtain it under the (heuristic) assumption that the square of the hull behaves as a random code. Still, this behavior is confirmed by numerical simulations.

Improving the algorithm with refinements Our basic algorithm can be improved by considering *refinements*, a concept that is somewhat similar to what the SSA already does. Starting from a given PEP instance, one can obtain several new PEP instances by applying code operations such as puncturing or shortening. These operations modify the code parameters and have an impact on the hull, as well. In this paper we consider only the case of puncturing and show how they can be used to obtain more convenient parameters for the squared hull (namely, a larger dimension of the dual). We claim that puncturing in a small number of positions is enough to reduce significantly the impact of false positives. Numerical simulations corroborate this fact.

The algorithm with refinements looks as in Figure 1. Taking everything into account, our approach can be used to solve PEP instances with hull dimension $h < \sqrt{2n} - 0.5 = \sqrt{2n} \cdot (1 + o(1))$. For these instances, on average, our algorithm runs in polynomial time.

To confirm the effectiveness of our algorithm, we have implemented our algorithm and ran intensive numerical simulations on random weakly-self dual codes with various parameters. All the code we have used is publicly available <https://github.com/psantini1992/SquareCodeEquiv>. For a big part of the paper, we focus on the case of low values of q since the impact of false positives is expected to be more significant in such a regime. For cryptographic applications, the required values of q are rather large and, in such cases, the impact of false positive is expected to be extremely mild.

Cryptanalysis of the instances from [ABL25] As a practical application of our algorithm, we consider the code equivalence instances recommended in [ABC⁺22]. Among other attacks, the hull dimension has been set so that both q^h and n^h (the times of SSA and the reduction to GIP, respectively) exceed 2^λ , with λ being the security parameter. However, the resulting instances have hull dimensions which are much less than $\sqrt{2n}$ and rather large values of q (e.g., $q \approx 2^{13}$): they constitute perfect use cases for our algorithm, which is expected to run in time $O(n^3)$. Notice that the instances from [ABL25] rely on the signed version of PEP: still, our algorithm can be applied to solve these instances, as well. This is possible since a signed permutation mapping two codes maps also the duals, hence one can just focus on the hulls. Squaring the hull flattens the scalar coefficients (since the square of ± 1 is 1) and one still ends up solving a

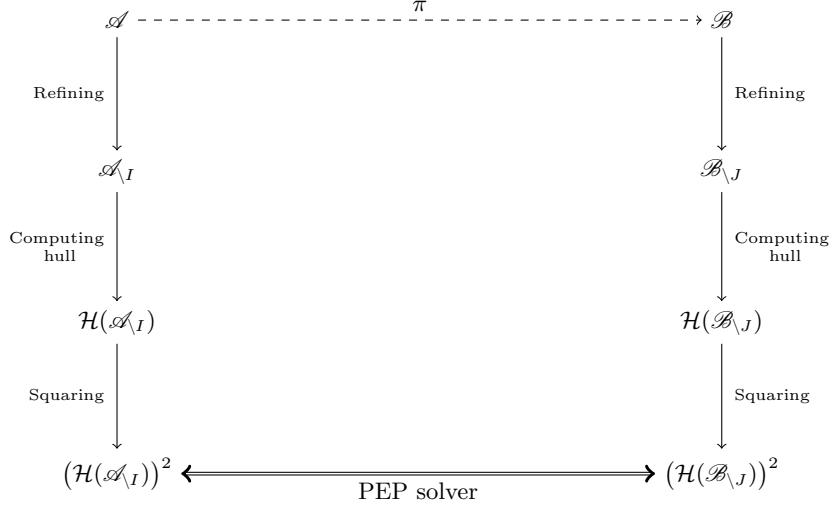


Fig. 1: Representation of how a PEP instance $(\mathcal{A}, \mathcal{B})$ can be transformed into a new PEP instance using the square code and refinements. In the figure, I and J are the sets used for the puncturing.

PEP instance. Once the permutation is found, retrieving the scalar coefficients boils down just to solve an overdetermined linear system.

We have tested our Sage implementation on the two parameters sets for 128 bits of security from [ABL25]. We have considered 100 instances for each parameter set and have always been able to correctly retrieve the secret permutation, after an average time of dozens of minutes. Notice that the scheme from [ABL25] can be easily made secure against our attack by choosing codes with larger hulls, say, a hull with dimension $h > \sqrt{2n}$.

1.2 Organization of the paper

In Section 2 we provide the notation used throughout the paper and the mathematical preliminaries. In Section 3 we describe and analyze our algorithm, based on using the Schur product to solve PEP. The analysis on the behaviour of the hull of the square of a self-orthogonal code is rather technical and is presented in Section 4. Finally, in Section 5 we describe the attack on the instances from [ABL25].

2 Preliminaries

In this section, we establish the notation we are going to use throughout the paper and recall useful background notions and facts about linear codes and the code equivalence problem.

2.1 Notation

Matrices and vectors Throughout the paper, q will denote a prime power, i.e., $q = p^m$ for a prime p and a positive integer m . We will use $\mathbb{F}_q$ to indicate the finite field with q elements and $\mathbb{F}_q^*$ to indicate its multiplicative group. Bold lower case (resp., upper case) will be used for vectors (resp., matrices). The general linear group formed by the non-singular $k \times k$ matrices over $\mathbb{F}_q$ is indicated as $\text{GL}_k(\mathbb{F}_q)$. The identity with size k is indicated as $\mathbf{I}_k$, while $\mathbf{0}$ denotes the null matrix (its dimensions will always be clear from the context). Sometimes, we indicate the set $\{1, \dots, n\}$ as $[n]$. For a set $J \subseteq [n]$ and a vector $\mathbf{a} \in \mathbb{F}_q^n$, we indicate $\mathbf{a}_J = (a_i)_{i \in J}$, i.e., $\mathbf{a}_J$ is the vector formed by the entries of $\mathbf{a}$ which are indexed by J . Analogously, $\mathbf{a}_{\setminus J} = \mathbf{a}_{\{1, \dots, n\} \setminus J} = (a_i)_{i \notin J}$.

Permutations We denote by S_n the symmetric group on n elements, and consider its elements as permutations of n objects. For a vector $\mathbf{a} = (a_1, \dots, a_n)$ and $\pi \in S_n$, we have

$$\pi(\mathbf{a}) = (a_{\pi^{-1}(1)}, \dots, a_{\pi^{-1}(n)}).$$

We naturally extend the action of permutations on matrices: $\pi(\mathbf{A})$ indicates the matrix resulting from the action of π on the columns of $\mathbf{A}$.

Sometimes we will represent a permutation π through the associated $n \times n$ matrix $\mathbf{P}$, so that $\pi(\mathbf{A}) = \mathbf{A}\mathbf{P}$; with some abuse of notation, we write $\mathbf{P} \in S_n$ if $\mathbf{P}$ is an $n \times n$ permutation matrix.

2.2 Linear codes

A linear code $\mathcal{C} \subseteq \mathbb{F}_q^n$ with length n and dimension k , sometimes indicated as an $[n, k]$ code, is a k -dimensional subspace of $\mathbb{F}_q^n$. The quantity $R = k/n$ is called *code rate*, and any vector $\mathbf{c} \in \mathcal{C}$ is called *codeword*. A canonical representation for a code is through a *generator matrix*, that is, a full-rank matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ such that $\mathcal{C} = \{\mathbf{u}\mathbf{G} \mid \mathbf{u} \in \mathbb{F}_q^k\}$.

The dual code $\mathcal{C}^\perp$ is the code comprising all vectors orthogonal to all the codewords of $\mathcal{C}$, namely: $\mathcal{C}^\perp = \{\mathbf{v} \in \mathbb{F}_q^n \mid \mathbf{c}\mathbf{v}^\top = 0, \forall \mathbf{c} \in \mathcal{C}\}$. The dual code has dimension $n - k$ and is generated by a full-rank matrix $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$, which is called *parity-check matrix*, for which $\mathbf{G}\mathbf{H}^\top = \mathbf{0}$.

Definition 1 (Hull) *The hull of a code is defined as the intersection between the code and its dual:*

$$\mathcal{H}(\mathcal{C}) = \mathcal{C} \cap \mathcal{C}^\perp.$$

We say that the hull of a code is trivial if it contains only the null vector.

The hull is, by definition, a subcode of both $\mathcal{C}$ and $\mathcal{C}^\perp$, thus has dimension at most $\min\{k, n - k\}$. As we will see in the next sections, the hull plays a crucial role in the complexity of many algorithms that solve PEP.

Definition 2 Let $\mathcal{C} \subseteq \mathbb{F}_q^n$ be a linear code with dimension k . We say that $\mathcal{C}$ is

- self-orthogonal if $\mathcal{C} \subseteq \mathcal{C}^\perp$
- self-dual if $\mathcal{C} = \mathcal{C}^\perp$
- weakly self-dual if $\mathcal{H}(\mathcal{C})$ is non trivial.

2.3 Random codes and density of weakly-self dual codes

By random code, we mean a code sampled uniformly at random among the $\begin{bmatrix} n \\ k \end{bmatrix}_q$ codes having length n and dimension k , over $\mathbb{F}_q$. Here,

$$\begin{bmatrix} n \\ k \end{bmatrix}_q = \frac{(q^n - 1)(q^{n-1} - 1) \cdots (q^{n-k+1} - 1)}{(q^k - 1)(q^{k-1} - 1) \cdots (q - 1)} = \prod_{i=0}^{k-1} \frac{q^{n-i} - 1}{q^{k-i} - 1}$$

denotes the Gaussian binomial.

As one would expect, the probability that a random code has hull dimension h decreases as h grows and thus (weakly) self-dual codes are rather sparse. This matter has been thoroughly studied in [Sen97]; here, we recall one of the main results of the paper.

Theorem 1 [SS13, Theorem 4] Let $\sigma_{n,k,h}$ indicate the number of weakly self-dual codes with hull dimension $h \geq 1$, over $\mathbb{F}_q$. When n and k goes to infinity, the ratio between $\sigma_{n,k,h}$ and $\begin{bmatrix} n \\ k \end{bmatrix}_q$ converges to ζ_h , where

$$\zeta_h = \gamma_q \cdot \prod_{i=1}^h \frac{1}{q^i - 1},$$

where $\gamma_q = \prod_{i=1}^{\infty} q^i \cdot \frac{q^i - 1}{q^{2i} - 1}$. In other words:

$$\lim_{\substack{n \rightarrow \infty \\ k \rightarrow \infty}} \frac{\sigma_{n,k,h}}{\begin{bmatrix} n \\ k \end{bmatrix}_q} = \zeta_h.$$

Remark 1 The quantity ζ_h can also be interpreted as the probability that a random $[n, k]$ code has hull dimension h . Notice that ζ_h does not depend on the code length nor on the code dimension, and decays as q and h grow. In particular, it decays exponentially with h^2 . To see this, we start by approximating γ_q as $1 - 1/q$ and further continue as

$$\zeta_h \approx (1 - 1/q) \cdot \prod_{i=1}^h \frac{1}{q^i - 1} \approx (1 - 1/q) \cdot \prod_{i=1}^h q^{-i} = (1 - 1/q) \cdot q^{-h(h+1)/2}.$$

When q is large, we further have $1 - 1/q \approx 1$, so we get a crude approximation of ζ_h as $\zeta_h \approx q^{-h(h+1)/2}$. For instance, for $h = 1$, we get $\zeta_1 \approx 1/q$, while for $h = 2$ we get $\zeta_2 \approx 1/q^3$. In particular, from [ABL25, Lemma 1], a random code has a trivial hull (i.e., with $h = 0$) with probability

$$\zeta_0 \geq 1 - 1/q - 1/q^2. \tag{1}$$

For large q , this can be further approximated as $1 - 1/q$.

2.4 Operations on codes

We recall some operations on code.

Definition 3 (Puncturing) *Given a linear code $\mathcal{C} \subseteq \mathbb{F}_q^n$ with dimension k , we define its puncturing in positions $J \in [n]$ as*

$$\mathcal{P}(\mathcal{C}, J) = \{(c_i)_{i \notin J} \mid (c_1, \dots, c_n) \in \mathcal{C}\}.$$

For the sake of simplicity, sometimes we will write $\mathcal{C}_{\setminus J}$ in place of $\mathcal{P}(\mathcal{C}, J)$. It can be seen that $\mathcal{C}_{\setminus J}$ has dimension at most k and length $n - |J|$.

Definition 4 (Shortening) *Given a linear code $\mathcal{C} \subseteq \mathbb{F}_q^n$ with dimension k , we define its shortening in positions $J \in [n]$ as*

$$\mathcal{S}(\mathcal{C}, J) = \mathcal{P}(\mathcal{C}^*, J),$$

where

$$\mathcal{C}^* = \{\mathbf{c} \in \mathcal{C} \mid \mathbf{c}_J = (0, \dots, 0)\}.$$

For the sake of simplicity, sometimes we will write $\mathcal{C}_{-J}$ in place of $\mathcal{S}(\mathcal{C}, J)$. It can be seen that $\mathcal{C}_{-J}$ has dimension at most k and length $n - |J|$.

Proposition 1 [HP10, Theorem 1.5.7] *Let $\mathcal{C}$ be a linear code with length n and $J \subseteq [n]$. Then*

- $\mathcal{P}(\mathcal{C}, J)^\perp = \mathcal{S}(\mathcal{C}^\perp, J)$;
- $\mathcal{S}(\mathcal{C}, J)^\perp = \mathcal{P}(\mathcal{C}^\perp, J)$.

When the hull of a code is non-trivial, shortening allows to decrease the hull dimension. In particular, if a constant number of positions are eliminated then the expected dimension is lowered at the cost of a multiplicative factor [Sen00].

Given two vectors $\mathbf{a} = (a_1, \dots, a_n), \mathbf{b} = (b_1, \dots, b_n) \in \mathbb{F}_q^n$, we define their *component-wise product*, denoted with the symbol $\star$, as follows:

$$\mathbf{a} \star \mathbf{b} := (a_1 b_1, \dots, a_n b_n).$$

This operation is also known as *Schur product* or *star product*. The component-wise product of vectors is clearly commutative and associative. We can thus use the notation $\star_{\mathbf{a} \in S} \mathbf{a} = \mathbf{a}_1 \star (\mathbf{a}_2 \star (\dots \star (\mathbf{a}_{s-1} \star \mathbf{a}_s)))$ for a finite set $S = \{\mathbf{a}_1, \dots, \mathbf{a}_s\}$, with the convention that $\star_{\mathbf{a} \in \emptyset} \mathbf{a} = (1, \dots, 1)$ and $\star_{\mathbf{a} \in \{\mathbf{x}\}} \mathbf{a} = \mathbf{x}$. The notion of component-wise product can then be extended to linear spaces over finite fields, i.e. to linear codes.

Definition 5 (Schur product of codes) *Let $\mathcal{A}, \mathcal{B} \subseteq \mathbb{F}_q^n$ be two linear codes with length n , their Schur product $\mathcal{A} \star \mathcal{B}$ is defined as*

$$\mathcal{A} \star \mathcal{B} = \text{span}(\{\mathbf{a} \star \mathbf{b} \mid \mathbf{a} \in \mathcal{A}, \mathbf{b} \in \mathcal{B}\}).$$

When the codes are the same, we speak of square code.

Definition 6 (Square code) Let $\mathcal{C} \subseteq \mathbb{F}_q^n$ be a linear code with length n , the square code $\mathcal{C}^2$ of $\mathcal{C}$ is defined as

$$\mathcal{C}^2 = \mathcal{C} \star \mathcal{C} = \text{span}(\{\mathbf{c} \star \mathbf{c}' \mid \mathbf{c}, \mathbf{c}' \in \mathcal{C}\}).$$

One can define iteratively the Schur product among more than two codes. In particular, we have the *powers of a code*:

$$\mathcal{C}^{i+1} = \mathcal{C}^i \star \mathcal{C}.$$

To avoid confusion with the classical operation of squaring a matrix, if $\mathbf{G}$ is the generator matrix of $\mathcal{C}$, we denote with $\mathbf{G}^{[2]}$ the generator matrix of $\mathcal{C}^2$.

The following lemma on the inner product between component-wise products of vectors will also come in handy thereafter.

Lemma 1 Let $S \subset \mathbb{F}_q^n$ be a set of vectors. Let $\{A, B\}$ and $\{C, D\}$ be two partitions of S . Then,

$$\langle \star_{\mathbf{a} \in A} \mathbf{a}, \star_{\mathbf{b} \in B} \mathbf{b} \rangle = \langle \star_{\mathbf{c} \in C} \mathbf{c}, \star_{\mathbf{d} \in D} \mathbf{d} \rangle.$$

Proof. Both sides of the equation are equal to $\sum_{j \in [n]} \prod_{\mathbf{s} \in S} \mathbf{s}_j$. □

We now give some results on how squaring a (weakly)-self-dual code affects both its dual and its hull.

We start with the simple case of $q = 2$.

Theorem 2 For any $\mathcal{C} \subseteq \mathbb{F}_2^n$, $\mathcal{C} \subseteq \mathcal{C}^2$ and $\mathcal{C}^{2\perp} \subseteq \mathcal{C}^\perp$.

Proof. For any vector $\mathbf{c} \in \mathbb{F}_2^n$, it holds that $\mathbf{c} \star \mathbf{c} = \mathbf{c}$. So, $\mathcal{C}$ is a subcode of $\mathcal{C}^2$. By orthogonality, $\mathcal{C}^{2\perp} \subseteq \mathcal{C}^\perp$. □

For codes having a sufficiently large dimension (say, larger than $\sqrt{2n}$), it is likely that the square code gives the full vector space $\mathbb{F}_q^n$. However, for self-orthogonal codes, this is not true anymore, as the next theorem shows.

Theorem 3 Let $\mathcal{C} \subseteq \mathbb{F}_q^n$ be a self-orthogonal code. Then, $(1, \dots, 1)$ is in the dual of $\mathcal{C}^2$.

Proof. Since the code is self-orthogonal, for any two codewords $\mathbf{a}, \mathbf{b} \in \mathcal{C}$, it holds that

$$0 = \sum_{i=1}^n a_i b_i = (\mathbf{a} \star \mathbf{b}) \cdot (1, 1, \dots, 1)^\top.$$

Since this holds for any pair of codewords $\mathbf{a}, \mathbf{b}$, then $(1, \dots, 1)$ is orthogonal to all the codewords of $\mathcal{C}^2$. □

The above theorem implies that the square code of a self-orthogonal is never full. Moreover, it also tells us that two self-orthogonal codes with sufficiently large dimensions can have the same square code. This will become important in order to study the performances of the solvers we discuss later.

2.5 Permutation Code Equivalence Problem

For a code $\mathcal{C} \subseteq \mathbb{F}_q^n$ and $\pi \in S_n$, we denote

$$\pi(\mathcal{C}) = \{\pi(\mathbf{a}) \mid \mathbf{a} \in \mathcal{C}\}.$$

In other words, $\pi(\mathcal{C})$ is the code obtained by applying π to all codewords of $\mathcal{C}$. It is easy to see that $\mathcal{C}$ and $\pi(\mathcal{C})$ have some common properties, such as the same minimum distance and the same weight distribution. For this reason, we say that $\mathcal{C}$ and $\pi(\mathcal{C})$ are *permutation equivalent* and write $\mathcal{C} \equiv_{S_n} \pi(\mathcal{C})$.

Problem 1 *Permutation Equivalence Problem (PEP)*

Given two linear codes $\mathcal{A}, \mathcal{B} \subseteq \mathbb{F}_q^n$, decide if they are permutation equivalent, i.e., if there exists $\pi \in S_n$ such that $\mathcal{B} = \pi(\mathcal{A})$. In other words, given two generator matrices $\mathbf{A}, \mathbf{B} \in \mathbb{F}_q^{k \times n}$, decide if there exists an invertible matrix $\mathbf{S} \in GL_k(\mathbb{F}_q)$ and a permutation matrix $\mathbf{P} \in S_n$ such that $\mathbf{B} = \mathbf{SAP}$.

In this paper we will sometimes focus on the search version of PEP, which asks to determine the permutation. According to the reduction in [BM23], the search and decisional versions are equivalent in terms of complexity.

Regarding equivalent codes and the Schur product, the following theorem holds.

Theorem 4 [Ran15, Lemma 2.49] *Let $\mathcal{B} = \pi(\mathcal{A})$ for some permutation $\pi \in S_n$. Then, $\mathcal{A}^2$ and $\mathcal{B}^2$ are linearly equivalent. In particular, $\mathcal{B}^2 = \pi(\mathcal{A}^2)$.*

Proof. Let us consider two codewords $\mathbf{a}, \mathbf{a}' \in \mathcal{A}$. Since $\mathcal{B} = \pi(\mathcal{A})$, there exist a pair of codewords $\mathbf{b}, \mathbf{b}' \in \mathcal{B}$ such that

$$\begin{aligned} \mathbf{b} &= (b_1, b_2, \dots, b_n) \\ &= (a_{\pi^{-1}(1)}, a_{\pi^{-1}(2)}, \dots, a_{\pi^{-1}(n)}), \end{aligned}$$

$$\begin{aligned} \mathbf{b}' &= (b'_1, b'_2, \dots, b'_n) \\ &= (a'_{\pi^{-1}(1)}, a'_{\pi^{-1}(2)}, \dots, a'_{\pi^{-1}(n)}). \end{aligned}$$

Let $\mathbf{c} = \mathbf{a} \star \mathbf{a}' \in \mathcal{A}^2$ and $\mathbf{c}' = \mathbf{b} \star \mathbf{b}' \in \mathcal{B}^2$. Observe that the j -th component of $\mathbf{c}$ is $a_j a'_j$, while the j -th component of $\mathbf{c}'$ is

$$b_j b'_j = a_{\pi^{-1}(j)} a'_{\pi^{-1}(j)} = c_{\pi^{-1}(j)}.$$

By linearity and considering all the components, we get $\mathcal{B}^2 \supseteq \pi(\mathcal{A}^2)$. The other inclusion is analogous. $\square$

2.6 Solvers for code equivalence

We briefly summarize the most important solvers for PEP. While more efficient solvers for relaxed instances exist, such as [BCDD⁺24, BN24], we only consider the standard case defined in Problem 1.

Algorithms based on low-weight codeword finding The first algorithm to solve the code equivalence was proposed by Leon in 1982 [Leo82]. The best generic solvers for this problem are ISD algorithms. Leon’s algorithm requires to find, for both codes $\mathcal{A}$ and $\mathcal{B}$, all codewords having weight not greater than some w which

- cannot be too large, otherwise finding these codewords takes too much time, and
- cannot be too small, otherwise the two sets do not possess enough structure and reconstructing the map between the codes is not possible.

We are not aware of any precise criterion to choose w properly; as a rule of thumb, many authors consider w to be slightly larger than the minimum distance d of the code. Given that finding low weight codewords is a notoriously hard problem, this solver runs in exponential time. An improvement to Leon’s algorithm has been proposed by Beullens in 2020 [Beu20]. This algorithm improves upon Leon’s since, when q is large enough, only a bunch of weight- w codewords is necessary; this yields a polynomial speed-up with respect to Leon’s algorithm but, in any case, even Beullens’s algorithm runs in exponential time.

The Support Splitting Algorithm Another algorithm to solve PEP is the Support Splitting Algorithm (SSA), proposed by Sendrier in 2000 [Sen00], which, in a nutshell, finds the unknown permutation by reconstructing its action index by index. This is obtained by using what Sendrier called *signature function*: a function s taking as input a code and an index $i \in \{1, \dots, n\}$ that is invariant under permutations, that is,

$$s(\mathcal{A}, i) = s(\mathcal{B}, \pi(i)), \quad \forall i \in \{1, \dots, n\}.$$

Given access to a *sufficiently discriminant* signature function (that is, a function such that $s(\mathcal{A}, i) \neq s(\mathcal{A}, j)$ for the wide majority of indices $i \neq j$), SSA can retrieve efficiently the secret permutation, by progressively filtering out all the pairs of indices which lead to different signatures. Eventually, a given signature function can be made more discriminant with the use of *refinements*.

Currently, the only known signature function requires to compute the weight enumerator function of the hull, which contains q^h codewords. Considering polynomial overheads, the algorithm has time complexity $O(n^3 + n^2 \log(n)q^h)$.

Reducing to Graph Isomorphism In 2019, Bardet et al. have proposed a reduction from PEP to GIP taking worst case time $O(n^{\omega+h})$, with ω being the exponent for matrix multiplication [BOST19]. When hulls are trivial, the reduction requires essentially to compute a bunch of matrix inversions and multiplications. Because of Babai’s algorithm [Bab16], we know that GIP can be solved in quasi-polynomial time, so that all PEP instances with trivial hull can be solved in worst case quasi-polynomial time, as well. The reduction takes more time when hull are non trivial but, as soon as $h = O(1)$ or $h = O(\log^a(n))$, the algorithm takes worst case quasi polynomial time.

Since in the experiments we ran for this paper we relied on this solver, we now recall the details of how it works.

For a code $\mathcal{A}$ with trivial hull and dimension k , any generator matrix $\mathbf{A}$ is such that $\text{Rank}(\mathbf{A}\mathbf{A}^\top) = k$. Then, one can associate $\mathcal{A}$ with a simple, undirected and weighted graph with the adjacency matrix

$$\mathbf{X} = \text{Adj}(\mathbf{A}) = \mathbf{A}^\top (\mathbf{A}\mathbf{A}^\top)^{-1} \mathbf{A} \in \mathbb{F}_q^{n \times n}.$$

The matrix $\mathbf{X}$ can be seen as the adjacency matrix for an undirected, weighted graph with edge labels given by finite field elements. It is easy to see that the function is invariant under changes of basis, i.e., $\mathbf{X} = \text{Adj}(\mathbf{A}) = \text{Adj}(\mathbf{S}\mathbf{A})$ for any $\mathbf{S} \in GL_k(\mathbb{F}_q)$.

Now, let $\mathbf{B} = \mathbf{S}\mathbf{A}\mathbf{P}$ with $\mathbf{S} \in GL_k(\mathbb{F}_q)$ and $\mathbf{P} \in S_n$ be a generator matrix for $\mathcal{B}$; to the code $\mathcal{B}$, we associate a graph through the function Adj . Notice that

$$\begin{aligned} \mathbf{Y} &= \text{Adj}(\mathbf{B}) = \mathbf{B}^\top (\mathbf{B}\mathbf{B}^\top)^{-1} \mathbf{B} \\ &= \mathbf{P}^\top \mathbf{A}^\top \mathbf{S}^\top (\mathbf{S}\mathbf{A}\mathbf{A}^\top \mathbf{S}^\top)^{-1} \mathbf{S}\mathbf{A}\mathbf{P} \\ &= \mathbf{P}^\top \mathbf{A}^\top \mathbf{S}^\top \mathbf{S}^{-\top} (\mathbf{A}\mathbf{A}^\top)^{-1} \mathbf{S}^{-1} \mathbf{S}\mathbf{A}\mathbf{P} \\ &= \mathbf{P}^\top \underbrace{\mathbf{A}^\top (\mathbf{A}\mathbf{A}^\top)^{-1} \mathbf{A}}_{\mathbf{X}} \mathbf{P}. \end{aligned}$$

Hence, the graphs whose adjacency matrices are $\mathbf{X}$ and $\mathbf{Y}$ are isomorphic and any permutation $\mathbf{P}$ so that $\mathbf{Y} = \mathbf{P}^\top \mathbf{X} \mathbf{P}$ gives also a solution to PEP.

If hulls are non trivial, then the above transformation cannot be applied because $\mathbf{A}\mathbf{A}^\top$ (and $\mathbf{B}\mathbf{B}^\top$) does not have full rank. Yet, reducing to GIP is still possible, even though the reduction gets more complicated. The authors in [BOST19] propose to handle this case by shortening $\mathcal{A}$ and $\mathcal{B}$ on sets I and J , respectively, which must be information sets for the hulls: the resulting codes have trivial hulls and adjacency matrices can be computed. The reduction works when the set I is mapped into J : this requires to enumerate, in the worst case, all candidates for J . For any pair I, J , the reduction requires other operations such as puncturing the codes and enumerating all permutations that send I to J . All in all, the reduction takes time $O(n^{\omega+h})$.

Algebraic solvers In [Sae17], the author discusses algebraic solvers for the code equivalence problem. Notice that precise complexity estimates are not given and the authors acknowledge that the proposed solver is efficient only when the code length is extremely small and the finite field size is small as well (say, $q \leq 5$).

3 Solving PEP with the Schur Product

In this section we identify a wide range of new PEP instances that can be solved in polynomial time. As a basis of our solvers, we have the following fact.

Fact 1 *Let $\mathcal{C} \subseteq \mathbb{F}_q^n$ be an $[n, k]$ code with hull dimension h , sampled uniformly at random, and let $\mathcal{C}^2$ be its square code. If $k \leq \sqrt{2n}$, then $\mathcal{H}(\mathcal{C}^2)$ is small with overwhelming probability.*

The justification of this result is definitely non trivial and, for this reason, we dedicate Section 4 to such a matter. In this section, we describe how the above result can be used to solve efficiently certain PEP instances.

3.1 Basic idea

Let us recall Theorem 4: if $\mathcal{A}$ and $\mathcal{B}$ are two equivalent codes, then their squares $\mathcal{A}^2$ and $\mathcal{B}^2$ are equivalent as well. In particular, if $\mathcal{B} = \pi(\mathcal{A})$ for some permutation $\pi \in S_n$, then $\mathcal{B}^2 = \pi(\mathcal{A}^2)$.

In some cases, solving the equivalence problem on the pairs $(\mathcal{A}^2, \mathcal{B}^2)$ may be easier than solving it on the initial pair $(\mathcal{A}, \mathcal{B})$. Indeed, let us consider a PEP instance $(\mathcal{A}, \mathcal{B})$ in which both codes are self-orthogonal. Recalling Fact 1, after the squaring, the hulls of $\mathcal{A}$ and $\mathcal{B}$ are small with large probability. This unlocks the possibility of using an efficient PEP solver on the pair $(\mathcal{A}^2, \mathcal{B}^2)$.

The main limitation is that this attack can be applied only when the square codes do not fill the space, i.e., when the dimensions of $\mathcal{A}^2$ and $\mathcal{B}^2$ are less than n . Indeed, if $\mathcal{A}^2 = \mathcal{B}^2 = \mathbb{F}_q^n$, then obviously the new instance is trivial, since any permutation would map a code into the other. In the following, we will call *degenerate* an $[n, n]$ code, i.e., a code that corresponds to the space $\mathbb{F}_q^n$. As we show later on, this requires to focus on codes having dimension which is either $\leq \sqrt{2n}$ or $\geq n - \sqrt{2n}$.

We furthermore extend the attack to the case of codes with any dimension (even greater than $\sqrt{2n}$ or less than $n - \sqrt{2n}$) but such that their hull has dimension $\leq \sqrt{2n}$. Basically, we do the attack by squaring only the hulls instead of the full codes.

False positives and spurious solutions As a little drawback of our algorithm, we have to deal with the potential existence of spurious solutions, that is, permutations that would solve $(\mathcal{A}^2, \mathcal{B}^2)$ but not $(\mathcal{A}, \mathcal{B})$. The existence of such solutions would fool our algorithm and, for this reason, we speak of “false positives”. Notice that the existence of false positives may be due to two different phenomena:

- the codes $\mathcal{A}$ and $\mathcal{B}$ are not permutation equivalent but their hulls are equivalent, i.e.,

$$\mathcal{A} \not\equiv_{S_n} \mathcal{B} \quad \text{but} \quad \mathcal{H}(\mathcal{A}) \equiv_{S_n} \mathcal{H}(\mathcal{B});$$

- the codes $\mathcal{A}$ and $\mathcal{B}$ are not permutation equivalent neither are their hulls, but the squares of the hulls are equivalent, i.e.,

$$\mathcal{A} \not\equiv_{S_n} \mathcal{B} \quad \text{and} \quad \mathcal{H}(\mathcal{A}) \not\equiv_{S_n} \mathcal{H}(\mathcal{B}), \quad \text{but} \quad (\mathcal{H}(\mathcal{A}))^2 \equiv_{S_n} (\mathcal{H}(\mathcal{B}))^2.$$

More generally, even when the initial codes are permutation equivalent, there may exist spurious solutions, that is, solutions that would solve PEP on $\mathcal{H}(\mathcal{A})^2$ and $\mathcal{H}(\mathcal{B})^2$. This may hinder solving search PEP.

Common sense suggests that such a situation happens very rarely. We show that, with some caution for what concerns some technical caveats, this is indeed the case: thanks to heuristic arguments, we are able to show that the existence of spurious solutions is extremely unlikely. Yet, when the squared hull has dimension $n - u$ with u too small, then false positives become rather likely and should be dealt with.

Refining the algorithm Similarly to what SSA does, we apply code operations (such as puncturing and shortening) to play a bit with the parameters of the square codes. Here, the goal is to reduce the impact of false positives and spurious solutions by enlarging the co-dimension u of the squared hull. When u is too small, one can puncture in a set of size z and reduce the hull dimension up to $h - z$: this increases the co-dimension of the squared hull. Thanks to the use of some heuristics, we argue that setting $z = O(\log_2(n))$ in the worst case is enough to make occurrence of false positive and spurious solutions very unlikely. In practice, with the help of numerical simulations, we show that even setting z to a very small constant (say, $z = 1$) is in many cases enough.

3.2 Non degenerate square codes

We first derive the conditions which guarantee that the square of a given $[n, k]$ code has dimension less than n . First, let $\mathbf{G}$ be the generator matrix for a code $\mathcal{C}$. Then, the generator matrix $\mathbf{G}^{[2]}$ of $\mathcal{C}^2$ satisfies the following properties:

Proposition 2 *If $\mathcal{C} \subseteq \mathbb{F}_q^n$ is a linear code with length n and dimension k , then $\mathcal{C}^2$ is a linear code with length n and dimension $k' \leq \min \left\{ n, \frac{k(k+1)}{2} \right\}$. More precisely, if $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ is a generator matrix for $\mathcal{C}$, then a generator matrix for $\mathcal{C}^2$ is a matrix $\mathbf{G}^{[2]}$ having as rows a maximal set of linearly independent vectors in the form*

$$\mathbf{g}_i \star \mathbf{g}_j$$

with $1 \leq i \leq j \leq k$, where $\mathbf{g}_i$ denotes the i -th row of $\mathbf{G}$.

Thus, to have that the dimension is less than n we ask that

$$\frac{k(k+1)}{2} < n,$$

which results in

$$k < k^* = \frac{-1 + \sqrt{1 + 8n}}{2}.$$

It is easy to see that

$$\sqrt{2n} - \frac{1}{2} < k^* < \sqrt{2n}$$

hence $k^* = \sqrt{2n} \cdot (1 + o(1))$. So, whenever $k < k^*$, a code is guaranteed to have a square which is not degenerate, i.e., the dimension of the square code is strictly less than n .

We can actually extend the range of PEP instances which can be solved using our algorithm with the trivial observation that, if $\mathcal{A} \equiv_{S_n} \mathcal{B}$, then also $\mathcal{A}^\perp \equiv_{S_n} \mathcal{B}^\perp$. So, the very same attack can be performed by looking at the duals. In such a case, obtaining a non degenerate square code is guaranteed if

$$n - k < k^* \implies k > n - k^*.$$

As we have already said, one can look only at the hulls. In such a case, the validity of our attack extends to all codes having any dimension k but hull of size $< \sqrt{2n}$, that is, all weakly self-dual codes with hull of dimension $h < \sqrt{2n}$.

3.3 Non trivial automorphisms as a cause of false positives

Estimating precisely the probability with which false positives occur is definitely a hard task. However, with the help of some heuristics and combinatorial arguments, we are able to say something about this matter. Interestingly, we find that the dimensions of the squared hulls should not be too large, otherwise the probability that our algorithm gets fooled becomes larger.

As a warm up, let us consider two self-orthogonal codes $\mathcal{A}, \mathcal{B} \subseteq \mathbb{F}_q^n$ where $k = h$ is such that $\frac{h(h+1)}{2} = n - 1$. For any pair $\mathcal{A}, \mathcal{B}$, we have $\mathcal{A}^2 = \mathcal{B}^2$. Indeed, recalling Theorem 3, we have $\mathcal{A}^{2\perp} = \mathcal{B}^{2\perp} = \text{Span}((1, \dots, 1))$. In such a case, our algorithm will always end up concluding that the two codes are equivalent.

Hence, one should have at least $\frac{h(h+1)}{2} \leq n - 2$. Actually, as we argue next, issues may happen whenever $\frac{h(h+1)}{2} = n - u$ with u too small.

First, consider the following theorem.

Theorem 5 *Let $\mathcal{A}, \mathcal{B} \subseteq \mathbb{F}_q^n$ be two random codes with dimension $u - 1$ for some positive integer $1 \leq u < n$ and such that $(1, \dots, 1) \in \mathbb{F}_q^n$ is contained in both $\mathcal{A}$ and $\mathcal{B}$. Then, the probability that $\mathcal{A} \equiv_{S_n} \mathcal{B}$ is not greater than $\frac{n!}{\left[\frac{n-1}{u-1} \right]_q}$. In particular, for constant q and for $u = 1 + c \log_2(n)$ for some constant $c > \frac{1}{\log_2(q)} \left(1 - \frac{\log_2(e)}{\log_2(n)} \right)$, the probability decays exponentially with n .*

Proof. For any $\mathcal{A}$, the number of codes $\mathcal{B}$ which are permutation equivalent to $\mathcal{A}$ cannot be greater than $n!$. Since the number of choices for $\mathcal{B}$ is $\left[\frac{n-1}{u-1} \right]_q$ [Sen97, Proposition 1], we obtain the probability in the thesis. We now study the asymptotic regime, assuming q is constant and n grows. To do this, we first

find an upper bound on $\left[\begin{smallmatrix} n-1 \\ u-1 \end{smallmatrix} \right]_q = \prod_{i=0}^{u-2} \frac{q^{n-1-i}-1}{q^{u-1-i}-1}$. For any $x > y$, it holds that $\frac{x-1}{y-1} > \frac{x}{y}$; in our case, $x := q^{n-1-i}$ and $y := q^{u-1-i}$ and $x/y = q^{n-u} > 1$ by hypothesis. So, we get

$$\left[\begin{smallmatrix} n-1 \\ u-1 \end{smallmatrix} \right]_q \geq \prod_{i=0}^{u-2} \frac{q^{n-1-i}}{q^{u-1-i}} = q^{(u-1)(n-u)}.$$

Now, we use Stirling's expansion for the factorial and thus bound the probability as

$$\frac{n!}{\left[\begin{smallmatrix} n-1 \\ u-1 \end{smallmatrix} \right]_q} \leq \frac{n!}{q^{(u-1)(n-u)}} \sim 2^{n \log_2(n/e) - (u-1)(n-u) \log_2(q)}.$$

We now take $u = 1 + c \log_2(n)$ for some constant $c > 0$, and further get

$$\begin{aligned} 2^{n \log_2(n/e) - (u-1)(n-u) \log_2(q)} &= 2^{n \cdot (\log_2(n/e) - (u-1) \log_2(q)(1-u/n))} \\ &= 2^{n \cdot (\log_2(n/e) - c \log_2(q) \log_2(n)(1-u/n))}. \end{aligned} \quad (2)$$

The term u/n grows as $\log_2(n)/n$ hence converges to 0 for growing n ; thus, the exponent of the above probability asymptotically converges to

$$n \cdot \underbrace{\log \left(n^{1-c \log_2(q)} \cdot e^{-1} \right)}_{\zeta_c(n, q)}.$$

Whenever $\zeta_c(n, q)$ is negative, the probability in Eq. (2) decreases exponentially with n . By setting $\zeta_c(n, q) < 0$, we obtain

$$n^{1-c \log_2(q)} \cdot e^{-1} < 1 \quad \implies \quad c > \frac{1}{\log_2(q)} \left(1 - \frac{\log_2(e)}{\log_2(n)} \right).$$

□

Remark 2 Observe that the constant c is always less than 1. For instance, if $c = 2/\log_2(q)$, then the probability to sample two permutation equivalent codes decays at least as $2^{-n \cdot \log_2(e \cdot n)}$. In particular, we have that u must be at least $1 + 2 \log_2(n)/\log_2(q)$.

We now use the above theorem to estimate the probability that our algorithm encounters a false positive. We do this with the (strong) heuristic assumption that squared hulls behave as random codes.

Heuristic 1 Let $\mathcal{A}, \mathcal{B} \subseteq \mathbb{F}_q^n$ be two random codes with hull dimension h , such that the squared hulls have dimension $n - u$. Then, if $u > 1 + \frac{\log_2(n)}{\log_2(q)}$, the probability to encounter false positives is negligible in n .

This heuristic shall be used only as a rule of thumb to determine how large u should be. In practice, we shall use u as large as $1 + \log_2(n)/\log_2(q)$ to expect that the algorithm can succeed with large probability. For instance, as we are going to see in the next sections, whenever $u = 2$ and $n > q$, our algorithm fails with rather large probability. Indeed, $n > q$ implies $\log_2(n) > \log_2(q)$ so we shall have

$$u > 1 + \log_2(n)/\log_2(q) > 2.$$

3.4 Spurious solution for search PEP

Even when not encountering false positives, having squared hulls with a dimension which is too large may lead to issues when solving PEP. Assume $\mathcal{B} = \pi(\mathcal{A})$ for some permutation π and that $\mathcal{H}(\mathcal{B})^2$ has non trivial permutation automorphisms. For each automorphism τ , we obtain a distinct solution to the PEP instance given by the squared hulls as $\pi \circ \tau^{-1}$. Obviously, the same applies to automorphisms of $\mathcal{H}(\mathcal{A})^2$.

Understanding how the automorphism group of the squared hulls behave is definitely a hard task and is out of the scope of this paper. Still, we are able to show that when u is too small, then automorphisms must necessarily exist. This is done in the following theorem.

Theorem 6 *Let $\mathcal{C} \subseteq \mathbb{F}_q^n$ be a self-orthogonal code with a square of dimension $n - u$. If $u < 1 + \log_q(n)$, then the permutation automorphism group of $\mathcal{C}^2$ is non trivial.*

Proof. Let $n - u$ indicate the dimension of $\mathcal{C}^2$, so that its dual $\mathcal{C}^{2\perp}$ has dimension u . Since the permutation automorphism group of a code corresponds to the permutation automorphism group of its dual, we can focus on $\mathcal{C}^{2\perp}$. From Theorem 3, we know that $(1, \dots, 1) \in \mathcal{C}^{2\perp}$. Let us consider a generator for $\mathcal{C}^{2\perp}$, which will be in the form:

$$\begin{pmatrix} 1 & 1 & \dots & 1 \\ \mathbf{a}_1 & \mathbf{a}_2 & \dots & \mathbf{a}_n \end{pmatrix},$$

where each $\mathbf{a}_i$ is a column vector of length $u - 1$. The maximum number of distinct vectors $\mathbf{a}_i$ we can have is q^{u-1} : whenever this value is less than n , then we have repeated entries. Any permutation that swaps repeated entries is an automorphism. $\square$

Even when $\mathcal{C}^{2\perp}$ is large enough so that the above inequality is not satisfied, it may statistically happen that automorphisms given by identical coordinates exist.

Proposition 3 *Let $\mathbf{A} \in \mathbb{F}_q^{(u-1) \times n}$ be a uniformly random matrix. Then, the probability it has all distinct columns is $\prod_{i=1}^{n-1} 1 - \frac{i}{q^{u-1}}$. When $n \ll q^{u-1}$, this is well approximated by $e^{\frac{1}{2}n^2 \cdot q^{u-1}}$.*

Proof. The probability simply corresponds to the probability that, when sampling uniformly at random n vectors from $\mathbb{F}_q$, they are all distinct. We now use the fact that, when $0 \leq x \ll 1$, it holds that $1 - x \approx e^{-x}$. Since $i < n < q^{u-1}$, then $1 - \frac{i}{q^{u-1}} \approx e^{\frac{-i}{q^{u-1}}}$; then

$$\prod_{i=1}^{n-1} 1 - \frac{i}{q^{u-1}} \approx e^{\frac{-\sum_{i=1}^{n-1} i}{q^{u-1}}} = e^{\frac{-n(n-1)}{2q^{u-1}}} \approx e^{\frac{-n^2}{2q^{u-1}}}.$$

□

Remark 3 Setting $u = 1 + c \log_2(n)$ for some constant $c > 2/\log_2(q)$, we obtain that the above probability grows as $e^{-\frac{1}{2}n^{-a}}$ for some $a > 0$. Asymptotically, this is $1 - \frac{1}{2}n^{-a} = O(1)$.

Observe that the previous analysis only tells us that u should be large enough to avoid that squared hulls have repeated coordinates. For what concerns the other automorphisms, we do not give any type of analysis. Still, heuristically, we expect that they exist with extremely small probability. Again, this intuition is confirmed by numerical simulations.

3.5 Using refinements

To reduce the impact of false positives and spurious solutions, one can apply some refinements on our algorithm. For instance, let $I, J \subseteq \{1, \dots, n\}$ such that J is the image of I under the action of π , i.e., for any index $i \in I$ it holds that $\pi(i) \in J$. Then, one can set $\mathcal{A}_I = \mathcal{P}(\mathcal{A}, I)$ and $\mathcal{B}_J = \mathcal{P}(\mathcal{B}, J)$ and then use our attack on the pair $(\mathcal{A}_I, \mathcal{B}_J)$. Whenever I and J are contained in at least one information set of the hulls, then the punctured codes will have hulls with dimension less than h . To this end, we consider the following theorem.

Theorem 7 Let $\mathcal{A} \subseteq \mathbb{F}_q^n$ be a self-orthogonal code with dimension h . Let $I \subseteq \{1, \dots, n\}$ with size z and such that there exists an information set containing I . Let $\mathcal{A}_I = \mathcal{P}(\mathcal{A}, I)$. Then, $\mathcal{A}_I$ has a hull with dimension $h - z$.

Proof. For the sake of simplicity and without loss of generality, we give the proof for the case $z = 1$ and $I = \{1\}$, assuming I is contained in the information set $\{1, \dots, h\}$. Let $\mathbf{G} \in \mathbb{F}_q^{h \times n}$ indicate the systematic generator matrix for $\mathcal{A}$. A generator for $\mathcal{A}_I$ is obtained by removing the first column of $\mathbf{G}$. Let $(g_{1,1}, g_{1,2}, \dots, g_{1,n})$ indicate the first row of $\mathbf{G}$. Then, the first row of the generator matrix of the punctured code is $(g_{1,2}, \dots, g_{1,n})$. Since $\mathcal{A}$ is self-orthogonal, we know that

$$0 = \sum_{j=1}^n g_{1,j}^2 = 1 + \sum_{j=2}^n g_{1,j}^2 \implies -1 = \sum_{j=2}^n g_{1,j}^2.$$

Hence, $(g_{1,2}, \dots, g_{1,n})$ is not self-orthogonal and cannot be in the hull. For the other $h - 1$ rows, puncturation removes a 0 hence all properties regarding orthogonality are preserved. □

Algorithm 1: Refined algorithm for solving PEP

Data: finite field size q , code length n , number z of punctured coordinates
Input: codes $\mathcal{A}, \mathcal{B} \subseteq \mathbb{F}_q^n$
Output: permutation $\pi \in S_n$ such that $\pi(\mathcal{A})\mathcal{B}$, or failure

```

/* Puncture on sets of  $z$  coordinates and solve search PEP */
1 for  $I \subseteq \{1, \dots, n\}, |I| = z$  do
2   Set  $\mathcal{A}_I = \mathcal{P}(\mathcal{A}, I)$ ;
3   for  $J \subseteq \{1, \dots, n\}, |J| = z$  do
4     Set  $\mathcal{B}_{\setminus J} = \mathcal{P}(\mathcal{B}, J)$ ;

    /* Solve search PEP on squared hulls of punctured codes */
5     Compute  $(\mathcal{H}(\mathcal{A}_I))^2, (\mathcal{H}(\mathcal{B}_{\setminus J}))^2$ ;
6     If it exists, find  $\tau \in S_{n-z}$  solving search PEP on  $(\mathcal{H}(\mathcal{A}_I))^2, (\mathcal{H}(\mathcal{B}_{\setminus J}))^2$ ;

    /* See if  $\tau$  leads to a solution for  $\mathcal{A}$  and  $\mathcal{B}$  */
7     Compute  $\mathbf{A} \in \mathbb{F}_q^{h \times (n-z)}$ , generator matrix for  $\mathcal{A}$ ;
8     Compute  $\mathbf{B} \in \mathbb{F}_q^{h \times (n-z)}$ , generator matrix for  $\mathcal{B}$ ;
9     Find  $\mathbf{S} \in GL_k(\mathbb{F}_q)$  such that  $\mathbf{S} \cdot \tau(\mathbf{A}_{\setminus I}) = \mathbf{B}_{\setminus J}$ ;
10    If it exists, find and return  $\pi \in S_n$  such that  $\pi(\mathbf{S} \cdot \mathbf{A}) = \mathbf{B}$ ;

/* No permutation has been found, codes are not equivalent */
11 return  $\perp$ 

```

The algorithm with refinements is given in Algorithm 1. Basically, the algorithm performs a brute force over all possible punctured patterns for both codes. For each pair (I, J) , the algorithm then solves search PEP on the punctured codes and, if it exists, finds a permutation $\tau \in S_{n-z}$. Then, one just needs to check whether this solution can be employed to obtain a solution for the initial instance $(\mathcal{A}, \mathcal{B})$. This can be done easily by working with generator matrices and employing trivial linear algebra.

Relying on puncturing, one can increase the co-dimension of the squared hull. In particular, working with the hulls of $\mathcal{A}_I$ and $\mathcal{B}_{\setminus J}$ always leads to a dual having larger dimension. Indeed, let $n' = n - z$ and $h' = h - z$ denote the code length and hull dimension, respectively, after puncturing. Let us indicate with $u = n - \frac{h(h+1)}{2}$ the (maximum) co-dimension without puncturation. The co-dimension of the squared hull after puncturation is

$$n' - \frac{h'(h'+1)}{2} = n - z - \frac{(h-z)(h-z+1)}{2} = n - \underbrace{\frac{h(h+1)}{2}}_u + \frac{2hz - z - z^2}{2}.$$

This is always greater than the dimension one would have without puncturing (which is u), since $2hz - z - z^2 \geq 0$ for any $0 \leq z \leq 2h - 1$.

Example 1 We make an explicit example of how refinements would work. Let us consider two weakly-self dual codes defined over $\mathbb{F}_{31}$, with length $n = 7$ and

dimension $k = h = 3$. Let the generator matrices of these codes be defined as

$$\mathbf{G} = \begin{pmatrix} 1 & 0 & 0 & 3 & 16 & 25 & 16 \\ 0 & 1 & 0 & 26 & 13 & 18 & 16 \\ 0 & 0 & 1 & 16 & 29 & 18 & 29 \end{pmatrix}, \quad \mathbf{G}' = \begin{pmatrix} 14 & 19 & 25 & 22 & 5 & 17 & 2 \\ 0 & 27 & 22 & 14 & 22 & 29 & 5 \\ 16 & 16 & 19 & 27 & 5 & 10 & 3 \end{pmatrix}.$$

The permutation sending the first code onto the second code is

$$\pi = (2, 6, 5, 4, 7, 3, 1),$$

i.e., the permutation sending the second coordinate to the first position, the sixth coordinates to the second positions, etc. The squares of these codes have dimension $\frac{h(h+1)}{2} = 6 = n-1$ but trivial hulls. Since we know that the all-ones codeword must be always be in the dual of a self-dual code, we know that

$$\mathcal{A}^{2\perp} = \mathcal{B}^{2\perp} = \text{Span}((1, \dots, 1)).$$

Any permutation would be a solution to PEP defined on $(\mathcal{A}^2, \mathcal{B}^2)$. We apply algorithm 1 using $I = \{1\}$ and $J = \{7\}$ (i.e., use $z = 1$). The hulls of the punctured codes $\mathcal{A}_{\setminus I}$ and $\mathcal{B}_{\setminus J}$ have dimension 2 and are generated by the following matrices

$$\mathbf{U} = \begin{pmatrix} 1 & 0 & 26 & 13 & 18 & 16 \\ 0 & 1 & 16 & 29 & 18 & 29 \end{pmatrix}, \quad \mathbf{U}' = \begin{pmatrix} 1 & 0 & 15 & 10 & 18 & 30 \\ 0 & 1 & 24 & 25 & 24 & 19 \end{pmatrix}.$$

We now consider the codes which are generated by the squares of $\mathbf{U}$ and $\mathbf{U}'$: the obtained matrices have trivial hull and have duals of dimension 3. Relying on the reduction to GIP, we obtain the two following adjacency matrices:

$$\mathbf{A} = \begin{pmatrix} 25 & 2 & 11 & 6 & 0 & 18 \\ 2 & 0 & 6 & 23 & 14 & 17 \\ 11 & 6 & 0 & 18 & 7 & 20 \\ 6 & 23 & 18 & 2 & 18 & 26 \\ 0 & 14 & 7 & 18 & 9 & 14 \\ 18 & 17 & 20 & 26 & 14 & 29 \end{pmatrix}, \quad \mathbf{A}' = \begin{pmatrix} 25 & 0 & 6 & 11 & 18 & 2 \\ 0 & 9 & 18 & 7 & 14 & 14 \\ 6 & 18 & 2 & 18 & 26 & 23 \\ 11 & 7 & 18 & 0 & 20 & 6 \\ 18 & 14 & 26 & 20 & 29 & 17 \\ 2 & 14 & 23 & 6 & 17 & 0 \end{pmatrix}.$$

We find that the permutation solving GIP is $\tau = (1, 5, 4, 3, 6, 2)$. With linear

algebra, we discover that $\mathbf{S} = \begin{pmatrix} 2 & 14 & 17 \\ 5 & 0 & 29 \\ 3 & 16 & 10 \end{pmatrix}$ is the 3×3 change of basis such that $\mathbf{S} \cdot \tau(\mathbf{G}_{\setminus I}) = \mathbf{G}'_{\setminus J}$. Now, we compute $\mathbf{S}\mathbf{G}$:

$$\mathbf{S}\mathbf{G} = \begin{pmatrix} 2 & 14 & 17 & 22 & 25 & 19 & 5 \\ 5 & 0 & 29 & 14 & 22 & 27 & 22 \\ 3 & 16 & 10 & 27 & 19 & 16 & 5 \end{pmatrix}.$$

Matching the columns of $\mathbf{S}\mathbf{G}$ with those of $\mathbf{G}'$, we reconstruct π .

3.6 Numerical simulations

We now show the effectiveness of our approach by reporting results of numerical simulations. We tested our attack on random PEP instances given by weakly self-dual codes, for several code parameters n , q and h .

As a solver for PEP, we have employed the reduction to GIP, relying on a very simple algorithm which reconstructs the permutation using the entries of rows of adjacency matrices as an invariant. Namely, let $\mathbf{X}, \mathbf{Y} \in \mathbb{F}_q^{n \times n}$ be two adjacency matrices: if row i of $\mathbf{X}$ and j of $\mathbf{Y}$ are the unique rows having the same entry multiset, then the permutation must send i to j . When a row matches with more than one row, then the algorithm fails. Notice that, obviously, there are much better algorithms for GIP. Still, even this basic solver is powerful enough when the finite field is sufficiently large: for $q \geq 5$ and under the heuristic assumption that the adjacency matrices we consider behave as random $n \times n$ matrices over $\mathbb{F}_q$, it has constant success probability. In particular, the success probability approaches 1 as soon as $q \geq 7$ and n grows.³ See Appendix A for details.

In Figure 2 we report results of numerical simulations about the success rate of our attack, considering random self-orthogonal codes defined over $\mathbb{F}_5$. In these simulations, we have $n = 30$ and several values of $k = h$. For all instances, we have considered two versions of our attack: the elementary one without refinements, and the one with refinements with $z = 1$ (i.e., puncturing on a single position). Observe that for a very small hull, our algorithm has very low success probability. For such cases, the algorithm from [BOST19] shall be employed. The success rate increases with growing h .

If we increase q , we expect that the success rate increases. This can be seen in Figures 3 and 4, in which we have $q = 7$ and $q = 61$, respectively. Going from $q = 5$ to $q = 7$ is already enough to increase significantly the success rate. Using $q = 61$, we see that even the basic version of our algorithm succeeds almost always, apart from the case $h = 7$. Using refinements, again, we obviously increase the success rates in all cases.

Observe that the co-dimension of the squared hull plays a crucial role in determining the success rate of our algorithm. For instance, for $h = 6$, we have $\frac{h(h+1)}{2} = 21$ hence the minimum co-dimension of the squared hull is $n - \frac{h(h+1)}{2} = 9$. This is much greater than $1 + 2 \log_2(n) / \log_2(q) \approx 3.9$ so we do not expect issues due to spurious solutions. Instead, for $h = 7$, we have $u = n - \frac{h(h+1)}{2} = 2$ and we see that, regardless of q , the basic algorithm always fails. Using refinements with $z = 1$, the algorithm has always correctly found the valid permutation. This is something we expected; indeed, for $z = 1$, the co-dimension given by

$$n - z - \frac{(h - z)(h - z + 1)}{2} = 29 - \frac{6 \cdot 7}{2} = 8,$$

³ This requires also to assume that the adjacency matrices we use in the reduction from PEP to GIP do not exhibit any “exotic” property. Indeed, these matrices are obtained with a very specific construction, starting from rectangular matrices which, in our case, are obtained by squaring $k \times n$ matrices.

which is much greater than $1 + 2\log_2(n)/\log_2(q)$.

Observe that, for large q , the algorithm succeeds with probability almost 1, apart from the cases in which the co-dimension is very small.

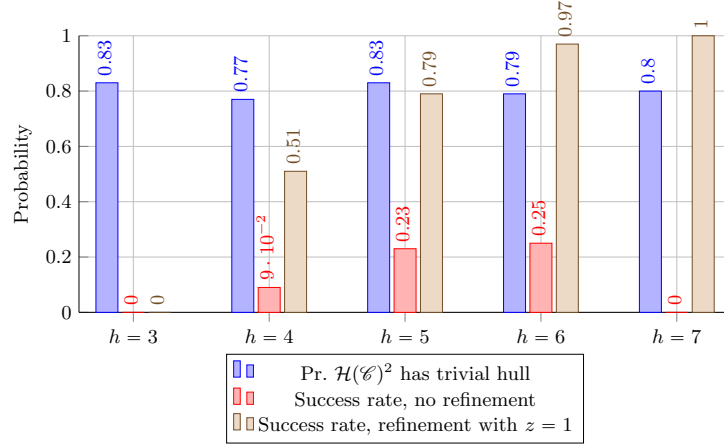


Fig. 2: Results of numerical simulation on self-orthogonal codes with $n = 30$ and $q = 5$, with several values of $k = h$. The success rate of the attack is evaluated only on the instances for which the square code has trivial hull. For each parameter set, we have generated 100 codes.

Finally, in Figure 5 we report about experiments conducted with growing values of n and self-orthogonal codes with $h = k = \lfloor \alpha\sqrt{n} \rfloor$, with $0 < \alpha \leq \sqrt{2}$. We have first considered the case of $\alpha = 1$, i.e., $h = \lfloor n \rfloor$. In such a case the co-dimension of the squared hull is approximately

$$n - \frac{\sqrt{n}(1 + \sqrt{n})}{2} \approx \frac{n}{2},$$

so we expect that false positives occur with very small probability. This is confirmed by numerical simulations (see Figure 5a). In any case, the success probability increases for growing n , apart from the case $q = 5$. In such a case, our solver for GIP is expected to have a constant success rate which is less than 1 (see Appendix A). Using refinements, the success rate increases significantly, even for $q = 5$ (see Figure 5b).

We have also considered the case of $h = \lfloor \sqrt{2n} - \frac{1}{2} \rfloor$, for which in many cases the co-dimension of the squared hull is very low. When this happens, the success probability of our algorithm drops (see Figure 5c). For instance, our basic version fails with large probability, regardless of q , for $n = 30$, for which $h = 7$ hence $u = n - \frac{h(h+1)}{2} = 2$. The success probability is low whenever the co-dimension of the squared hull is low. In any case, refinements can greatly boost the algorithm and raise the success probability, even for the case $q = 5$ (see Figure 5d).

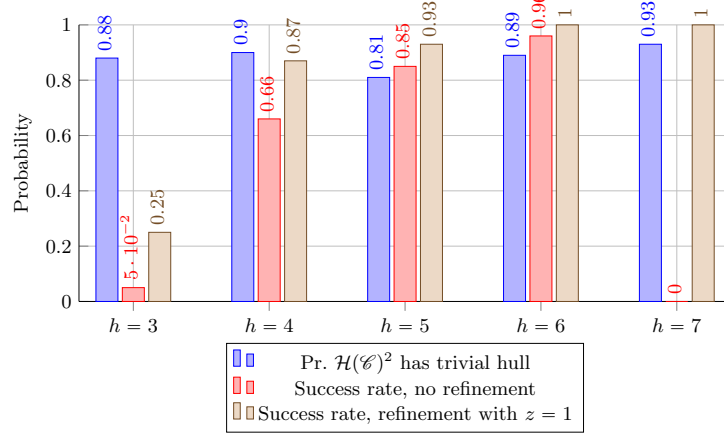


Fig. 3: Results of numerical simulation on self-orthogonal codes with $n = 30$ and $q = 7$, with several values of $k = h$. The success rate of the attack is evaluated only on the instances for which the square code has trivial hull. For each parameter set, we have generated 100 codes.

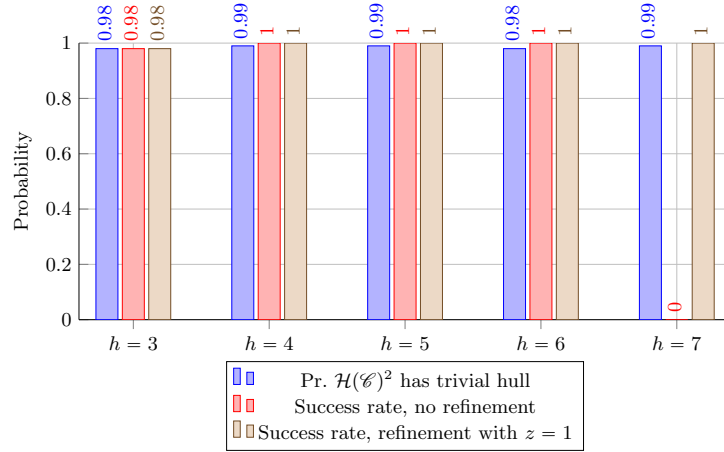


Fig. 4: Results of numerical simulation on self-orthogonal codes with $n = 30$ and $q = 61$, with several values of $k = h$. The success rate of the attack is evaluated only on the instances for which the square code has trivial hull. For each parameter set, we have generated 100 codes.

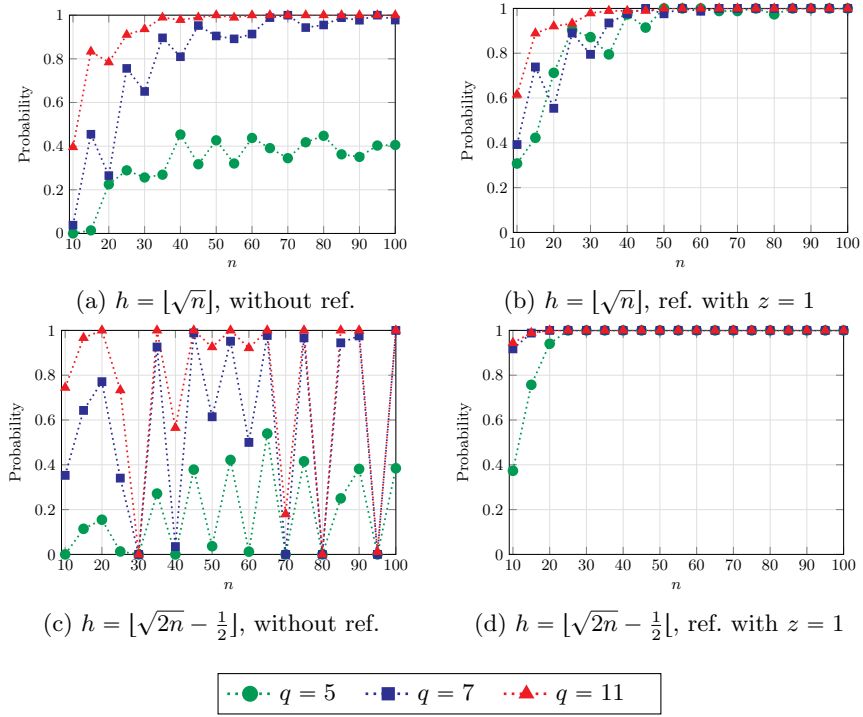


Fig. 5: Success rates of our algorithm for several combinations of $(q, n, k = h)$. Each data point has been estimated by simulating the attack on 100 codes.

4 The hull of the square of self-orthogonal codes

We discuss here the validity of Fact 1, which provide a theoretical basis for the attack to PEP for self-orthogonal codes.

Given two $[n, k]$ codes $\mathcal{A}, \mathcal{B} \subseteq \mathbb{F}_q^n$ and their respective generator matrices

$$\mathbf{A} = \begin{pmatrix} \mathbf{a}_1 \\ \vdots \\ \mathbf{a}_k \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \mathbf{b}_1 \\ \vdots \\ \mathbf{b}_k \end{pmatrix},$$

let us consider the $k \times k$ matrix

$$\mathbf{AB}^\top = (\langle \mathbf{a}_i, \mathbf{b}_j \rangle)_{\substack{1 \leq i \leq k \\ 1 \leq j \leq k}} \quad (3)$$

of inner products among the basis elements.

Theorem 8 *Let $\mathcal{A}, \mathcal{B} \subseteq \mathbb{F}_q^n$ be two $[n, k]$ linear codes with generator matrices $\mathbf{A}, \mathbf{B}$ respectively. Then*

$$\text{rank}(\mathbf{AB}^\top) = k - \dim_{\mathbb{F}_q} \mathcal{A} \cap \mathcal{B}^\perp.$$

Proof. A vector $\mathbf{y}$ belongs to $\mathcal{A} \cap \mathcal{B}^\perp$ if and only if it can be written as $\mathbf{y} = \sum_{i=1}^k \alpha_i \mathbf{a}_i$ for some $\alpha_i \in \mathbb{F}_q$ and $\langle \mathbf{y}, \mathbf{b}_i \rangle = 0$ for all $i = 1, \dots, k$. Let $\ker(\cdot)$ denote the right kernel. Given $\mathbf{y} = \sum_{i=1}^k \alpha_i \mathbf{a}_i \in \mathcal{A}$, we have that $(\alpha_1, \dots, \alpha_k)^\top \in \ker(\mathbf{AB}^\top)$ if and only if

$$0 = \mathbf{GG}'^\top \cdot \begin{pmatrix} \alpha_1 \\ \vdots \\ \alpha_k \end{pmatrix} = \begin{pmatrix} \langle \mathbf{y}, \mathbf{b}_1 \rangle \\ \vdots \\ \langle \mathbf{y}, \mathbf{b}_k \rangle \end{pmatrix},$$

i.e. if and only if $\mathbf{y} \in \mathcal{A} \cap \mathcal{B}^\perp$. It follows that

$$\dim_{\mathbb{F}_q} \ker(\mathbf{AB}^\top) = \dim_{\mathbb{F}_q} \mathcal{A} \cap \mathcal{B}^\perp.$$

Since $\mathbf{AB}^\top$ is a square matrix, we conclude that

$$\text{rank}(\mathbf{AB}^\top) = k - \dim_{\mathbb{F}_q} \ker(\mathbf{AB}^\top) = k - \dim_{\mathbb{F}_q} \mathcal{A} \cap \mathcal{B}^\perp.$$

Remark 4 *Theorem 8 implies that, despite $\mathbf{AB}^\top$ depends on the basis elements (and even on their order), the rank is invariant and only depends on the codes $\mathcal{A}, \mathcal{B}$.*

By taking $\mathcal{A} = \mathcal{B}$, we immediately get

Corollary 1 *Let $\mathcal{A} \subseteq \mathbb{F}_q^n$ be an $[n, k]$ linear code with generator matrix $\mathbf{A}$. Then*

$$\text{rank}(\mathbf{AA}^\top) = k - \dim_{\mathbb{F}_q} \mathcal{H}(\mathcal{A}).$$

We remark that $\mathbf{A}\mathbf{A}^\top$ is a symmetric matrix.

Consider now a linear self-orthogonal code $\mathcal{C} \subseteq \mathbb{F}_q^n$ of dimension k such that $\binom{k+1}{2} < n$ and with generator matrix $\mathbf{G}$. For simplicity, we assume that the dimension of its square code attains the upper bound, i.e. $\dim_{\mathbb{F}_q} \mathcal{C}^2 = \binom{k+1}{2}$, and define its basis $\{\mathbf{c}_{i,j} \mid 1 \leq i \leq j \leq k\}$, where $\mathbf{c}_{i,j} = \mathbf{c}_i \star \mathbf{c}_j$.

The matrix $\mathbf{G}\mathbf{G}^\top$ is the zero matrix, since $\mathcal{C}$ is self-orthogonal, and thus all the inner products $\langle \mathbf{c}_i, \mathbf{c}_j \rangle$ are null. We are rather interested in the matrix

$$\mathbf{M} \stackrel{\text{def}}{=} \mathbf{G}^{[2]} \mathbf{G}^{[2]\top} \in \mathbb{F}_q^{d \times d}, \quad (4)$$

as $\mathcal{C}^2$ potentially breaks the self-orthogonality. Such a matrix has several properties depending on the field over which it is defined. We will use the following notation to denotes entries in $\mathbf{M}$:

$$\mathbf{M}_{(i_1, i_2), (j_1, j_2)} = \langle \mathbf{c}_{i_1, i_2}, \mathbf{c}_{j_1, j_2} \rangle = \langle \mathbf{c}_{i_1} \star \mathbf{c}_{i_2}, \mathbf{c}_{j_1} \star \mathbf{c}_{j_2} \rangle.$$

Proposition 4 *Let $\mathcal{C} = \langle \mathbf{c}_1, \dots, \mathbf{c}_k \rangle \subseteq \mathbb{F}_q^n$ be an $[n, k]$ self-orthogonal code with $\binom{k+1}{2} < n$, and $\mathbf{M} = \mathbf{G}^{[2]} \mathbf{G}^{[2]\top}$. Then:*

1. for all $i_1, i_2, j_1, j_2 \in [1, k]$, $\mathbf{M}_{(i_1, i_2), (j_1, j_2)} = \mathbf{M}_{(i_1, j_1), (i_2, j_2)}$;
2. if $q = 2^s$, then all the diagonal entries are null, i.e. for all $i, j \in [1, k]$, $\mathbf{M}_{(i, j), (i, j)} = 0$;
3. if $q = 2^s$, then for all $i, j \in [1, k]$, $\mathbf{M}_{(i, i), (j, j)} = 0$;
4. if $q = 2$ or $q = 3$, then for all $i, j \in [1, k]$, $\mathbf{M}_{(i, i), (i, j)} = 0$.

Proof. Property 1 is obtained directly from Lemma 1 by taking two different partitions with blocks of cardinality 2 of a set of 4 elements.

Property 2 is obtained from Lemma 1 in the following way:

$$\langle \mathbf{c}_{i,j}, \mathbf{c}_{i,j} \rangle = \langle \mathbf{c}_i \star \mathbf{c}_j, \mathbf{c}_i \star \mathbf{c}_j \rangle = \langle \mathbf{c}_i^2, \mathbf{c}_j^2 \rangle = \langle \mathbf{c}_i, \mathbf{c}_j \rangle^2 = 0.$$

Property 3 readily follows as a corollary by combining Property 1 with $i_1 = i_2 = i$, $j_1 = j_2 = j$ together with Property 2.

Finally, Property 4 results from Lemma 1 as

$$\langle \mathbf{c}_{i,i}, \mathbf{c}_{i,j} \rangle = \langle \mathbf{c}_i \star \mathbf{c}_i, \mathbf{c}_i \star \mathbf{c}_j \rangle = \langle \mathbf{c}_i^3, \mathbf{c}_j \rangle = \langle \mathbf{c}_i, \mathbf{c}_j \rangle = 0.$$

Due to Theorem 8, one can study the distribution of the hull dimension for the square of a self-orthogonal code from the rank of the corresponding matrix (4). We recall here some classic results from [Mac69], which count the number of symmetric matrices over a finite field of a given rank.

Proposition 5 [Mac69, Theorems 2 & 3] *Let $N(t, r)$ denote the number of symmetric matrices of size $t \times t$, rank $r \leq t$, with entries in a finite field $\mathbb{F}_q$. Let $N_0(t, r)$ denote the number of symmetric matrices of size $t \times t$, rank $r \leq t$, with entries in a finite field $\mathbb{F}_q$, $q = 2^m$, and 0 on the main diagonal. Then*

$$\begin{aligned} N(t, 2s) &= \prod_{i=1}^s \frac{q^{2i}}{q^{2i}-1} \cdot \prod_{i=0}^{2s-1} (q^{t-i} - 1), \\ N(t, 2s+1) &= \prod_{i=1}^s \frac{q^{2i}}{q^{2i}-1} \cdot \prod_{i=0}^{2s} (q^{t-i} - 1), \\ N_0(t, 2s) &= \prod_{i=1}^s \frac{q^{2i-2}}{q^{2i}-1} \cdot \prod_{i=0}^{2s-1} (q^{t-i} - 1), \\ N_0(t, 2s+1) &= 0. \end{aligned}$$

Remark 5 Proposition 5 already explains a very particular behavior, that is the constant parity of the hull dimension for fields of characteristic 2 (for codes $\mathcal{C}$ such that $\dim_{\mathbb{F}_q} \mathcal{C}^2 = \binom{k+1}{2}$). Indeed, similarly as what happens for skew-symmetric matrices over fields of characteristic $\neq 2$, symmetric matrices over $\mathbb{F}_{2^m}$ with 0's on the main diagonal can only have even rank. Hence, by Theorem 8, the hull dimension must have the same parity as $\binom{k+1}{2}$, i.e. it is odd if $k \equiv 1 \pmod{4}$ or $k \equiv 2 \pmod{4}$ and it is even if $k \equiv 3 \pmod{4}$ or $k \equiv 0 \pmod{4}$.

Dividing the numbers $N(t, r)$ (resp. $N_0(t, r)$) by the total number of symmetric matrices, i.e. $q^{\binom{t+1}{2}}$ (resp. $q^{\binom{t}{2}}$), provides the relative frequency of a certain rank.

However, Proposition 4 shows that the matrix (4) corresponding to the square of a self-orthogonal code is not randomly distributed among all symmetric matrices. Instead, some very specific linear relations among its entries hold. Moreover, in the ternary and characteristic 2 cases, it is somewhat sparser. This means that such matrices depend on less than $\binom{k+1}{2} + 1$ parameters. The next example illustrates this behavior.

Example 2 We provide the generic matrix $\mathbf{M}$ for several parameters. Potentially different values in $\mathbb{F}_q$ are denoted by different symbols.

– Let $k = 3, q = 2$. Then

$$\begin{array}{cccccc} c_{1,1} & c_{1,2} & c_{2,2} & c_{1,3} & c_{2,3} & c_{3,3} \\ \left(\begin{array}{cccccc} 0 & 0 & 0 & 0 & * & 0 \\ 0 & 0 & 0 & * & * & * \\ 0 & 0 & 0 & * & 0 & 0 \\ 0 & * & * & 0 & * & 0 \\ * & * & 0 & * & 0 & 0 \\ 0 & * & 0 & 0 & 0 & 0 \end{array} \right) \begin{array}{l} c_{1,1} \\ c_{1,2} \\ c_{2,2} \\ c_{1,3} \\ c_{2,3} \\ c_{3,3} \end{array} \end{array}$$

– Let $k = 3, q = 3$. Then

$$\begin{array}{cccccc} c_{1,1} & c_{1,2} & c_{2,2} & c_{1,3} & c_{2,3} & c_{3,3} \\ \left(\begin{array}{cccccc} 0 & 0 & + & 0 & \# & @ \\ 0 & + & 0 & \# & \% & * \\ + & 0 & 0 & \% & 0 & ? \\ 0 & \# & \% & @ & * & 0 \\ \# & \% & 0 & * & ? & 0 \\ @ & * & ? & 0 & 0 & 0 \end{array} \right) \begin{array}{l} c_{1,1} \\ c_{1,2} \\ c_{2,2} \\ c_{1,3} \\ c_{2,3} \\ c_{3,3} \end{array} \end{array}$$

– Let $k = 3, q = 2^m, m > 1$. Then

$$\begin{matrix} & \mathbf{c}_{1,1} & \mathbf{c}_{1,2} & \mathbf{c}_{2,2} & \mathbf{c}_{1,3} & \mathbf{c}_{2,3} & \mathbf{c}_{3,3} \\ \begin{pmatrix} 0 & * & 0 & - & \# & 0 \\ * & 0 & \$ & \# & \% & ? \\ 0 & \$ & 0 & \% & ; & 0 \\ - & \# & \% & 0 & ? & \& \\ \# & \% & ; & ? & 0 & ! \\ 0 & ? & 0 & \& & ! & 0 \end{pmatrix} \end{matrix} \begin{matrix} \mathbf{c}_{1,1} \\ \mathbf{c}_{1,2} \\ \mathbf{c}_{2,2} \\ \mathbf{c}_{1,3} \\ \mathbf{c}_{2,3} \\ \mathbf{c}_{3,3} \end{matrix}$$

– Let $k = 4, q = 2$. Then

$$\begin{matrix} & \mathbf{c}_{1,1} & \mathbf{c}_{1,2} & \mathbf{c}_{2,2} & \mathbf{c}_{1,3} & \mathbf{c}_{2,3} & \mathbf{c}_{3,3} & \mathbf{c}_{1,4} & \mathbf{c}_{2,4} & \mathbf{c}_{3,4} & \mathbf{c}_{4,4} \\ \begin{pmatrix} 0 & 0 & 0 & 0 & * & 0 & 0 & 0 & + & - & 0 \\ 0 & 0 & 0 & * & * & * & + & + & ! & + & \\ 0 & 0 & 0 & * & 0 & 0 & + & 0 & ? & 0 & \\ 0 & * & * & 0 & * & 0 & - & ! & - & - & \\ * & * & 0 & * & 0 & 0 & ! & ? & ? & ? & \\ 0 & * & 0 & 0 & 0 & 0 & - & ? & 0 & 0 & \\ 0 & + & + & - & ! & - & 0 & + & - & 0 & \\ + & + & 0 & ! & ? & ? & + & 0 & ? & 0 & \\ - & ! & ? & - & ? & 0 & - & ? & 0 & 0 & \\ 0 & + & 0 & - & ? & 0 & 0 & 0 & 0 & 0 & \end{pmatrix} \end{matrix} \begin{matrix} \mathbf{c}_{1,1} \\ \mathbf{c}_{1,2} \\ \mathbf{c}_{2,2} \\ \mathbf{c}_{1,3} \\ \mathbf{c}_{2,3} \\ \mathbf{c}_{3,3} \\ \mathbf{c}_{1,4} \\ \mathbf{c}_{2,4} \\ \mathbf{c}_{3,4} \\ \mathbf{c}_{4,4} \end{matrix}$$

Consider, for instance, the case $q = 2, k = 3$. The generic matrix depends only on one parameter and its rank is therefore either 0 (when $*$ = 0) or 6 when (when $*$ = 1). From our experiments, the hull distribution splits accordingly in approximately 1/2 of the instances having hull dimension $6 - 6 = 0$ and the other half having hull dimension $6 - 0 = 6$. Proposition 5 remains, however, very useful to estimate the rank (and consequently the hull dimension) distribution for larger dimensions and/or field sizes. We tested this theoretical hull dimension distribution X against the experimental complementary rank distribution Y of matrices obtained from random self-orthogonal codes (over 10^4 samples), and we obtained a statistical distance $\frac{1}{2} \sum_{d \geq 0} |\mathbb{P}(X = d) - \mathbb{P}(Y = d)| < 0.01$ whenever

- $q = 2$ and $k \geq 8$;
- $q = 3$ and $k \geq 5$;
- $q = 4, 8, 9$ and $k \geq 4$;
- any k , otherwise.

The fact that the estimation gets better by letting the dimension grow is not surprising, as in this case the number of parameters on which the matrix depends increases as well, making singular behaviors less probable. More precisely, we can upper bound the number of parameters on which the generic matrix depends, based on the relations provided by Proposition 4.

Corollary 2 *Let $\mathbf{M}$ be the generic matrix constructed from an $[n, k]_q$ self-orthogonal code with $\binom{k+1}{2} < n$. Then:*

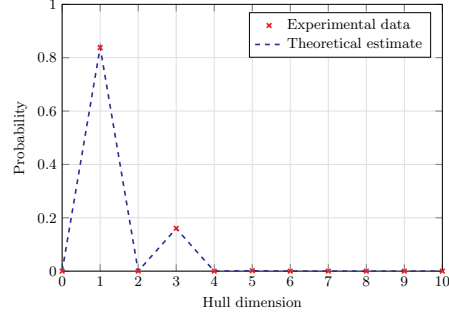


Fig. 6: Hull distribution for $q = 2, k = 9, n = 100$. The red plot shows the experimental distribution over 10000 tests while the blue plot is the estimation from Proposition 5.

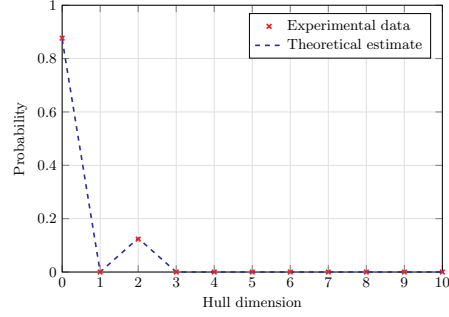


Fig. 7: Hull distribution for $q = 8, k = 4, n = 100$.

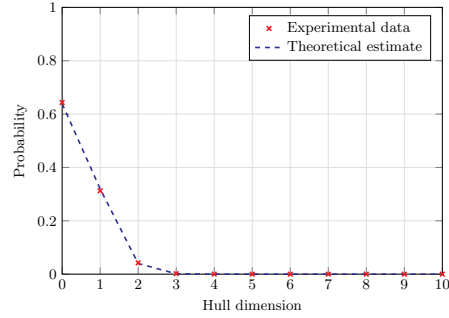


Fig. 8: Hull distribution for $q = 3, k = 6, n = 100$.

- if $q > 3$ odd, then $\mathbf{M}$ depends on at most $\binom{k+3}{4}$ parameters;
- if $q = 3$, then $\mathbf{M}$ depends on at most $\binom{k}{4} + 3\binom{k}{3} + \binom{k}{2}$ parameters;
- if $q = 2^m$, $m > 1$, then $\mathbf{M}$ depends on at most $\binom{k}{4} + 3\binom{k}{3} + 2\binom{k}{2}$ parameters;
- if $q = 2$, then $\mathbf{M}$ depends on at most $\binom{k}{4} + \binom{k}{3}$ parameters.

In all cases, the number of parameters is upper bounded by $\frac{k^4}{24}(1+o(1))$ as $k \rightarrow \infty$.

Proof. Property 1 of Proposition 4 associates entries $\mathbf{M}_{(i_1, i_2), (j_1, j_2)}$ to the multiset $\{i_1, i_2, j_1, j_2\}$ in such a way that if two entries have the same multiset then they are equal. The number of multisets of size 4 of the set $[1, k]$ is $\binom{k+3}{4}$.

If $q = 3$, Property 4 implies that multisets of the form $\{a, a, a, a\}$ or $\{a, a, a, b\}$, $b \neq a$ are associated with null entries. Hence, the number of parameters is at most

$$\binom{k+3}{4} - 2\binom{k}{2} - k = \binom{k}{4} + 3\binom{k}{3} + \binom{k}{2}.$$

If $q = 2^m$, $m > 1$, Property 2 implies that multisets of the form $\{a, a, b, b\}$ are associated with null entries. Hence, the number of parameters is at most

$$\binom{k+3}{4} - \binom{k}{2} - k = \binom{k}{4} + 3\binom{k}{3} + 2\binom{k}{2}.$$

Finally, if $q = 2$, Properties 2 and 4 implies that all the subsets with at at most 2 distinct elements are associated with null entries. Moreover, multisets with only three distinct elements are associated with entries with the same value if the corresponding sets are the same. Hence, the number of parameters coincides at most with the number of subsets of 3 or 4 elements:

$$\binom{k}{4} + \binom{k}{3}.$$

To summarize, the following heuristic on the hull dimension of the square of a self-orthogonal code, which is based on the previous explanations and experimental results, is enough to ensure that our algorithm works in polynomial time with high probability.

Heuristic 2 Let $\mathcal{C} \subseteq \mathbb{F}_q^n$ be an $[n, k]$ self-orthogonal code such that $\dim_{\mathbb{F}_q} \mathcal{C}^2 = \binom{k+1}{2}$. For $k, n \rightarrow \infty$ and $k < \sqrt{2n}(1+o(1))$, the probability that $\mathcal{H}(\mathcal{C}^2)$ has dimension 0 can be estimated by

$$\begin{aligned} & - \frac{N(\binom{k+1}{2}, \binom{k+1}{2})}{q^{\binom{k+1}{2}+1}}, \quad \text{if } q \text{ is odd;} \\ & - \frac{N_0(\binom{k+1}{2}, \binom{k+1}{2})}{q^{\binom{k+1}{2}}}, \quad \text{if } q = 2^m \text{ and } (k \equiv 3 \pmod{4} \text{ or } k \equiv 0 \pmod{4}). \end{aligned}$$

If $q = 2^m$ and $(k \equiv 1 \pmod{4} \text{ or } k \equiv 2 \pmod{4})$, the probability that $\mathcal{H}(\mathcal{C}^2)$ has dimension 1 can be estimated by

$$\frac{N_0(\binom{k+1}{2}, \binom{k+1}{2} - 1)}{q^{\binom{k+1}{2}}},$$

where the values of N and N_0 are given in Proposition 5.

5 Cryptanalysis applications

In this section, we show how the theory we have developed in this paper can be used to devise a polynomial time attack on the recommended instances of the Updatable Public-Key Encryption by Albrecht, Benčina and Lai presented at Eurocrypt 2025 [ABL25].

5.1 The UPKE scheme [ABL25]

The paper [ABL25] presents an Updatable Public-Key Encryption (UPKE) scheme whose security as a PKE relies on the Learning With Errors (LWE) assumption [Reg09]. In a nutshell, a UPKE scheme consists of 5 algorithms (KeyGen, Enc, Dec, UpdPk, UpdSk) such that

- (KeyGen, Enc, Dec) is a PKE scheme;
- any sender, including an adversary, can update another user’s public key pk by running the algorithm UpdPk, obtaining a new public key pk' and an update token up ;
- the receiver can use up to update their secret key sk to sk' by running the algorithm UpdSk.

UPKE schemes have been introduced as a slight relaxation of forward secure public key encryption as a remedy against the poor efficiency and the requirement that parties stay synchronized of the latter construction. A UPKE scheme is considered secure if an exposed secret key sk' does not reveal the message encrypted with any previous public key, under the assumption that at least one update in the chain leading to sk' was not malicious. We refer the reader to [DKW21] for a more in-depth discussion of the topic and for the technical definitions of correctness and IND-CR-CPA Security.

Previous post-quantum secure attempts of UPKE scheme are also based on LWE, but they suffer from a growing noise issue, thus they only support a bounded number of updates. On the opposite, the scheme’s update mechanism of [ABL25] has no practical upper bound on the number of updates. This is achieved thanks to the use of the signed permutation equivalence problem (SPEP) assumption, defined here below, in the random oracle model.

We first define $SP_n = S_n \ltimes \{\pm 1\}^{*n}$ as the group of signed permutation matrices, i.e. the matrices that can be written as $\mathbf{PD}$, where $\mathbf{P} \in S_n$ and $\mathbf{D}$ is a signature matrix, i.e. a diagonal matrix whose diagonal elements are 1 or -1, over $\mathbb{F}_q$.

Using signed permutation as isometries for the Hamming metric, we get the following problem.⁴

Problem 2 Signed Permutation Equivalence Problem (SPEP)

Given two linear codes $\mathcal{A}, \mathcal{B} \subseteq \mathbb{F}_q^n$, decide if $\mathcal{A} \equiv_{SP_n} \mathcal{B}$, i.e., if there exists

⁴ The main reason to use signed permutation in the context of lattices is that they are also isometries for the Euclidean metric.

λ	$\log_2(q)$	n	k	h
128	13	7313	450	27
128	16	11000	550	26
192	18	20250	900	37
256	19	29688	1250	48

Table 1: Parameters of the instances from [ABL25]

$\mu \in SP_n$ such that $\mathcal{B} = \mu(\mathcal{A})$. In other words, given two generator matrices $\mathbf{A}, \mathbf{B} \in \mathbb{F}_q^{k \times n}$, decide if there exist an invertible matrix $\mathbf{S} \in GL_k(\mathbb{F}_q)$ and a signed permutation matrix $\mathbf{M} \in SP_n$ such that $\mathbf{B} = \mathbf{SAM}$.

This variant of the code equivalence problem can be seen in a straightforward way as an intermediate version between PEP and LEP.

In [ABL25], the application of a uniformly random signed permutation to the key pair in the update mechanism does not provide any increase of noise, thus making the updating “free”. More in detail, a uniformly random tall matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times k}$, representing a dual-Regev public key [GPV08] and where q is a prime, is seen as the generator matrix (in column shape) of a linear code over $\mathbb{Z}_q$. This code is sampled in such a way that the hull dimension h is sufficiently large to make algorithms like SSA [Sen00] and the reduction to HIP [BOST19] not efficient. In particular, h is such that $\min(q^h, n^h) \geq 2^\lambda$, for a given security parameter λ .

The recommended parameters are reported in Table 1; the security parameter is indicated with λ .

5.2 Cryptanalysis

We observe that, in all the recommended instances, we have always $h \leq \sqrt{2n}$. This opens up for the possibility to apply our attack.

As a little technicality, we have to deal with the fact that the scheme employs SPEP. However, this comes with very little effort. As showed in [DG23], it is easy to see that if $\mathcal{B} = \mu(\mathcal{A})$ with μ being a signed permutation, then $\mathcal{H}(\mathcal{B}) = \mu(\mathcal{H}(\mathcal{A}))$. Indeed, for μ a signed permutation, it holds that $\mathcal{B}^\perp = \mu(\mathcal{A}^\perp)$. To see this, let us express μ in matrix form as $\mathbf{PD}$ with $\mathbf{P}$ a permutation matrix and $\mathbf{D}$ a diagonal matrix with diagonal taking values in $\{\pm 1\}$. Let $\mathbf{G}$ denote a generator matrix for $\mathcal{A}$; then, a generator matrix for $\mathcal{B}$ is $\mathbf{G}' = \mathbf{GPD}$. For $\mathbf{H}$ a parity-check matrix for $\mathcal{A}$, we claim that a parity-check matrix for $\mathcal{B}$ is $\mathbf{H}' = \mathbf{HPD}$. Indeed, it holds that

$$\mathbf{G}'\mathbf{H}'^\top = \mathbf{GP} \underbrace{\mathbf{D}^2}_{\mathbf{I}_n} \mathbf{P}^\top \mathbf{H}^\top = \mathbf{GPP}^\top \mathbf{H}^\top = \mathbf{0}.$$

In the above equation, we used the fact that $\mathbf{D}^2 = \mathbf{I}_n$, since $(\pm 1)^2 = 1$.

Let $\mu := (\mathbf{t}, \pi)$ with $\mathbf{t} \in \{\pm 1\}^n$. The permutation π can be found running our PEP attack on $(\mathcal{H}(\mathcal{A}))^2$ and $(\mathcal{H}(\mathcal{B}))^2$ since these two codes are only permutation equivalent and not signed permutation equivalent. Indeed, they would be mapped by the monomial represented by $\mathbf{P}\mathbf{D}^2$ but, as we have already seen, $\mathbf{D}^2 = \mathbf{I}_n$. Using a PEP solver on $(\mathcal{H}(\mathcal{A}))^2$ and $(\mathcal{H}(\mathcal{B}))^2$, we can find π .

We now have to find $\mathbf{D}$. Let us denote by $\mathbf{P}$ be the matrix associated to π : to recover $\mathbf{D}$, it is enough to compute a parity-check matrix $\mathbf{H}'$ for $\mathcal{B}$ and then set up a linear system of the form

$$\mathbf{G}\mathbf{P}\mathbf{D}\mathbf{H}'^\top = \mathbf{0}.$$

Let us indicate as $(t_1, \dots, t_n)$ the elements in the diagonal of $\mathbf{D}$, and treat them as unknowns. Let $\tilde{\mathbf{G}} = \mathbf{G}\mathbf{P} = \{\tilde{g}_{i,j}\}_{\substack{1 \leq i \leq k \\ 1 \leq j \leq n}}$; then, the above equation can be rewritten as

$$\left\{ \sum_{\ell=1}^n t_\ell \cdot \tilde{g}_{i,\ell} h'_{j,\ell} = 0 \right\}_{\substack{1 \leq i \leq k \\ 1 \leq j \leq n-k}} \implies \left\{ \tilde{\mathbf{g}}_i \star \mathbf{h}'_j \cdot \mathbf{t}^\top = 0 \right\}_{\substack{1 \leq i \leq k \\ 1 \leq j \leq n-k}}, \quad (5)$$

where $\tilde{\mathbf{g}}_i$ is the i -th row of $\tilde{\mathbf{G}}$ and $\mathbf{h}'_j$ is the j -th row of $\mathbf{H}'$.

The vector $\mathbf{t}$ lies in right kernel of the matrix $\{\tilde{\mathbf{g}}_i \star \mathbf{h}'_j\}_{\substack{1 \leq i \leq k \\ 1 \leq j \leq n-k}}$ which, with overwhelming probability, has dimension 1 and is hence generated by a unique vector which must be a scalar multiple of $\mathbf{t}$. In the unlikely case in which the kernel has dimension > 1 , we enumerate all vectors in the kernel, searching for all non-null vectors containing either one or two values. Scaling these vectors yields the secret $\mathbf{t}$. Moreover, taking a random subset of slightly more than n out of the $k(n-k)$ $\tilde{\mathbf{g}}_i \star \mathbf{h}'_j$ is sufficient in practice to solve the system with overwhelming probability.

The attack runs obviously in polynomial time, as all required operations are essentially basic linear algebra plus solving the PEP instance $(\mathcal{H}(\mathcal{A}))^2, (\mathcal{H}(\mathcal{B}))^2$ which, thanks to the analysis in this paper, is easy on average since the codes have trivial hull with large probability. As a crude estimate, we can assess the complexity of the whole attack as $O(n^3 \log_2(q))$, using $O(n^3 \log_2(q))$ as the cost of the reduction to GIP.

Testing the attack We have tested the attack, using the reduction to GIP as a solver for PEP. To solve GIP, we relied on the basic Algorithm 2 given in Appendix A.

As a sanity check, we have implemented our attack on the instances for 128 bits of security, recommended in [ABL25], using Sagemath.⁵ For each parameter set, we have tried the attack on 100 randomly generated instances of Signed PEP. Notice that in [ABL25] the authors do not give exact value for q : in our tests, we set q as the largest prime having which is less than $2^{\log_2(q)}$.

⁵ The code is available: <https://github.com/psantini1992/SquareCodeEquiv>

Since $n - \frac{h(h+1)}{2}$ is always much greater than $1 + \log_2(n)/\log_2(q)$, according to the analysis in Section 3 it is very unlikely that false positives or spurious solutions happen. For this reason, we ran the algorithm without refinements.

The attack has been tested on a standard laptop with a 13th Gen Intel(R) Core(TM) i7-1355U, with 16 GB RAM and running the Ubuntu 22.04.5 LTS operating system. For all tested instances, the squared hulls had trivial hulls and our PEP solver on $(\mathcal{H}(\mathcal{A}))^2$ and $(\mathcal{H}(\mathcal{B}))^2$ has correctly retrieved the secret key. To reduce the size of $\{\tilde{\mathbf{g}}_i \star \mathbf{h}'_j\}_{\substack{1 \leq i \leq k \\ 1 \leq j \leq n-k}}$, in our code we took random generator

$\tilde{\mathbf{G}}$ and parity-check $\mathbf{H}'$ matrices and selected a subset I for row-indexes i of cardinality slightly more than $\sqrt{nk/(n-k)}$. For each index i we selected a subset J_i for row-indexes j of cardinality slightly more than $\sqrt{n(n-k)/k}$ so that the total number of selected pairs (i, j) is slightly more than n . Alternatively, one can take $\tilde{\mathbf{G}}$ and parity-check $\mathbf{H}'$ matrices in systematic form and exploit the fact that the linear system is already partially reduced. In this case, however, one should pay attention to select, for any $1 \leq \ell \leq n$, at least one pair (i, j) such that $\tilde{g}_{i,\ell} h'_{j,\ell} \neq 0$, so that the ℓ -th entry of $\mathbf{t}$ appears in the system. In all experiments, the vector $\mathbf{t}$ has been always correctly found.

The attack ran required, on average, 643 seconds for the first instance.

Countermeasure Notice that this weakness can be easily avoided by enlarging the hull dimension of the codes employed in the scheme from [ABL25]. A good solution may be that of choosing h to be larger than $\sqrt{2n}$. Values of h that are too close to $\sqrt{2n}$ shall be avoided since, using refinements on a sufficiently large number of positions, one may end up again with codes having a hull which is small enough to apply our attack. Indeed, if $h \approx \sqrt{2n} + z$, then one can apply our attack using, as refinements, puncturations on approximately z positions. This would take lead to an overall cost of $O\left(\binom{n}{z} n^3 \log_2(q)\right)$. We recommend that z is set large enough, say, so that $\binom{n}{z} > 2^\lambda$.

Notice that, to be as conservative as possible, one may also rely on self-dual codes, for which $h = k$.

Acknowledgements

Authors wish to thank Martin R. Albrecht, Benjamin Benčina, and Russell W. F. Lai, for the feedback and the valuable discussion about the implementation of the UPKE algorithm.

The first and third authors are partially supported by the Italian Ministry of University and Research (MUR) under the PRIN 2022 program with projects “Mathematical Primitives for Post Quantum Digital Signatures” (CUP I53D23006580001) and “Post quantum Identification and eNcryption primitives: dEsign and Realization (POINTER)” (CUP I53D23003670006), by project SERICS (PE00000014) under the MUR National Recovery and Resilience Plan, funded by the European Union - Next Generation EU and by MUR under the

Italian Fund for Applied Science (FISA 2022), Call for tender No. 1405 published on 13-09-2022 - project title “Quantum-safe cryptographic tools for the protection of national data and information technology assets” (QSAFEIT) - No. FISA 2022-00618 (CUP I33C24000520001), Grant Assignment Decree no. 15461 adopted on 02.08.2024.

References

- ABC⁺22. Martin R Albrecht, Daniel J Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo Von Maurich, Rafael Misoczki, Ruben Niederhagen, et al. Classic mceliece: conservative code-based cryptography. 2022.
- ABL25. Martin Albrecht, Benjamin Benčina, and Russell WF Lai. Hollow lwe: A new spin, unbounded updatable encryption from lwe and pce. In *EUROCRYPT 2025*. 2025.
- Bab16. László Babai. Graph isomorphism in quasipolynomial time. In *Proceedings of the forty-eighth annual ACM symposium on Theory of Computing*, pages 684–697, 2016.
- BC14. Mikhail A Borodin and Ivan V Chizhov. Effective attack on the mceliece cryptosystem based on reed-muller codes. *Discrete Mathematics and Applications*, 24(5):273–280, 2014.
- BCDD⁺24. Alessandro Budroni, Jesús-Javier Chi-Domínguez, Giuseppe D’Alconzo, Antonio J. Di Scala, and Mukul Kulkarni. Don’t use it twice! solving relaxed linear equivalence problems. In *Advances in Cryptology – ASIACRYPT 2024: 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9–13, 2024, Proceedings. Part VIII*, page 35–65, Berlin, Heidelberg, 2024. Springer-Verlag.
- Beu20. Ward Beullens. Not enough less: An improved algorithm for solving code equivalence problems over $\mathbb{F}_q$. In *International Conference on Selected Areas in Cryptography*, pages 387–403. Springer, 2020.
- BM23. Jean-François Biasse and Giacomo Micheli. A search-to-decision reduction for the permutation code equivalence problem. In *2023 IEEE International Symposium on Information Theory (ISIT)*, pages 602–607. IEEE, 2023.
- BMT23. Magali Bardet, Rocco Mora, and Jean-Pierre Tillich. Polynomial time key-recovery attack on high rate random alternant codes. *IEEE Transactions on Information Theory*, 70(6):4492–4511, 2023.
- BN24. Alessandro Budroni and Andrea Natale. On the sample complexity of linear code equivalence for all code rates. Cryptology ePrint Archive, Paper 2024/1757, 2024.
- BOST19. Magali Bardet, Ayoub Otmani, and Mohamed Saeed-Taha. Permutation code equivalence is not harder than graph isomorphism when hulls are trivial. In *2019 IEEE International Symposium on Information Theory (ISIT)*, pages 2464–2468. IEEE, 2019.
- CDAG20. Alain Couvreur, Thomas Debris-Alazard, and Philippe Gaborit. On the hardness of code equivalence problems in rank metric. *arXiv preprint arXiv:2011.04611*, 2020.
- CGGU⁺14. Alain Couvreur, Philippe Gaborit, Valérie Gauthier-Umaña, Ayoub Otmani, and Jean-Pierre Tillich. Distinguisher-based attacks on public-key

- cryptosystems using reed-solomon codes. *Designs, Codes and Cryptography*, 73:641–666, 2014.
- CL21. Alain Couvreur and Matthieu Lequesne. On the security of subspace subcodes of reed-solomon codes for public key encryption. *IEEE Transactions on Information Theory*, 68(1):632–648, 2021.
- CMCP17. Alain Couvreur, Irene Márquez-Corbella, and Ruud Pellikaan. Cryptanalysis of mceliece cryptosystem based on algebraic geometry codes and their subcodes. *IEEE Transactions on Information Theory*, 63(8):5404–5418, 2017.
- CNP⁺23. Tung Chou, Ruben Niederhagen, Edoardo Persichetti, Tovohery Haja-tiana Randrianarisoa, Krijn Reijnders, Simona Samardjiska, and Monika Trimoska. Take your meds: digital signatures from matrix code equivalence. In *International Conference on Cryptology in Africa*, pages 28–52. Springer, 2023.
- COT16. Alain Couvreur, Ayoub Otmani, and Jean-Pierre Tillich. Polynomial time attack on wild mceliece over quadratic extensions. *IEEE Transactions on Information Theory*, 63(1):404–427, 2016.
- CPS25. Tung Chou, Edoardo Persichetti, and Paolo Santini. On linear equivalence, canonical forms, and digital signatures. *Designs, Codes and Cryptography*, pages 1–43, 2025.
- CPTZ25. Alain Couvreur, Rakhi Pratihari, Nihan Tanisah, and Ilaria Zappatore. On the structure of the schur squares of twisted generalized reed-solomon codes and application to cryptanalysis. In *International Conference on Post-Quantum Cryptography*, pages 3–34. Springer, 2025.
- DBB18. Vlad Drăgoi, Valeriu Beiu, and Dominic Bucerzan. Vulnerabilities of the mceliece variants based on polar codes. In *International Conference on Security for Information Technology and Communications*, pages 376–390. Springer, 2018.
- DG23. Léo Ducas and Shane Gibbons. Hull attacks on the lattice isomorphism problem. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *Public-Key Cryptography – PKC 2023*, pages 177–204, Cham, 2023. Springer Nature Switzerland.
- DKW21. Yevgeniy Dodis, Harish Karthikeyan, and Daniel Wichs. Updatable public key encryption in the standard model. In *Theory of Cryptography: 19th International Conference, TCC 2021, Raleigh, NC, USA, November 8–11, 2021, Proceedings, Part III 19*, pages 254–285. Springer, 2021.
- DvW22. Léo Ducas and Wessel van Woerden. On the lattice isomorphism problem, quadratic forms, remarkable lattices, and cryptography. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 643–673. Springer, 2022.
- FGUO⁺13. Jean-Charles Faugere, Valérie Gauthier-Umana, Ayoub Otmani, Ludovic Perret, and Jean-Pierre Tillich. A distinguisher for high-rate mceliece cryptosystems. *IEEE Transactions on Information Theory*, 59(10):6830–6844, 2013.
- GPV08. Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *Proceedings of the fortieth annual ACM symposium on Theory of computing*, pages 197–206, 2008.
- HP10. W Cary Huffman and Vera Pless. *Fundamentals of error-correcting codes*. Cambridge university press, 2010.

- Leo82. Jeffrey Leon. Computing automorphism groups of error-correcting codes. *IEEE Transactions on Information Theory*, 28(3):496–511, 1982.
- Mac69. Jessie MacWilliams. Orthogonal matrices over finite fields. *The American Mathematical Monthly*, 76(2):152–164, 1969.
- McE78. Robert J McEliece. A public-key cryptosystem based on algebraic. *Coding Thv*, 4244(1978):114–116, 1978.
- MT23. Rocco Mora and Jean-Pierre Tillich. On the dimension and structure of the square of the dual of a goppa code. *Designs, Codes and Cryptography*, 91(4):1351–1372, 2023.
- Now25. Julian Nowakowski. An improved algorithm for code equivalence. In *International Conference on Post-Quantum Cryptography*, pages 71–103. Springer, 2025.
- PR97. Erez Petrank and Ron M Roth. Is code equivalence easy to decide? *IEEE Transactions on Information Theory*, 43(5):1602–1604, 1997.
- Ran15. Hugues Randriambololona. On products and powers of linear codes under componentwise multiplication. *Algorithmic arithmetic, geometry, and coding theory*, 637:3–78, 2015.
- Reg09. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009.
- RS09. L.B. Richmond and Jeffrey Shallit. Counting abelian squares. *The Electronic Journal of Combinatorics [electronic only]*, 16(1):Research Paper R72, 9 p.–Research Paper R72, 9 p., 2009.
- RST24. Krijn Reijnders, Simona Samardjiska, and Monika Trimoska. Hardness estimates of the code equivalence problem in the rank metric. *Designs, Codes and Cryptography*, pages 1–30, 2024.
- Sae17. Mohamed Ahmed Saeed. *Algebraic approach for code equivalence*. PhD thesis, Normandie Université; University of Khartoum, 2017.
- Sen97. Nicolas Sendrier. On the dimension of the hull. *SIAM Journal on Discrete Mathematics*, 10(2):282–293, 1997.
- Sen00. Nicolas Sendrier. Finding the permutation between equivalent linear codes: The support splitting algorithm. *IEEE Transactions on Information Theory*, 46(4):1193–1203, 2000.
- SS13. Nicolas Sendrier and Dimitris E Simos. The hardness of code equivalence over and its application to code-based cryptography. In *International Workshop on Post-Quantum Cryptography*, pages 203–216. Springer, 2013.
- Wie10. Christian Wieschebrink. Cryptanalysis of the niederreiter public key scheme based on grs subcodes. In *Post-Quantum Cryptography: Third International Workshop, PQCrypto 2010, Darmstadt, Germany, May 25-28, 2010. Proceedings 3*, pages 61–72. Springer, 2010.

A A basic solver for the Graph Isomorphism Problem

The algorithm we consider in this paper to solve the search version of GIP is given in Algorithm 2. The algorithm uses, as a subroutine, a function `Count` that takes a tuple of n finite field elements $(a_1, \dots, a_n)$ and returns a length- q vector of integers whose i -th entry corresponds to the number of occurrences of the i -th finite field element. This can be implemented in time $O(n \log_2(q))$.

The algorithm receives as input adjacency matrices $\mathbf{X}, \mathbf{Y} \in \mathbb{F}_q^n$ for two weighted, undirected graphs and aims to find, if it exists, a permutation $\mathbf{P} \in S_n$

Algorithm 2: Simple algorithm to solve search GIP

Data: finite field size q , code length n
Input: adjacency matrices $\mathbf{X}, \mathbf{Y} \in \mathbb{F}_q^{n \times n}$
Output: permutation $\mathbf{P} \in S_n$, if it exists, such that $\mathbf{P}^\top \mathbf{X} \mathbf{P} = \mathbf{Y}$, or failure $\perp$ otherwise

```

1 Set  $\mathbf{P} = \mathbf{0}$ ; // null matrix

// Compute list with multiset entries for rows of  $\mathbf{X}$ 
2 Initialize  $\mathcal{L} = \emptyset$ ;
3 for  $i = 1, \dots, n$  do
4   Compute  $\mathbf{a}^{(i)} = \text{Count}(x_{i,1}, \dots, x_{i,n})$ ;
5   Update  $\mathcal{L} \leftarrow \mathcal{L} \cup \{(i, \mathbf{a}^{(i)})\}$ ;

// Compute multiset entries for rows of  $\mathbf{Y}$ , search for collisions
6 for  $j = 1, \dots, n$  do
7   Compute  $\mathbf{b}^{(j)} = \text{Count}(y_{j,1}, \dots, y_{j,n})$ ;
8   Find  $I \subseteq \{1, \dots, n\}$  such that, for each  $i \in I$ ,  $(i, \mathbf{a}^{(i)}) \in \mathcal{L}$  and  $\mathbf{a}^{(i)} = \mathbf{b}^{(j)}$ ;
9   if  $|I| \neq 1$  then
10    return  $\perp$ ;
11   else
12    Let  $I = \{i\}$ ; set  $p_{i,j} = 1$ ;

// Check if  $\mathbf{P}$  is a permutation matrix, report failure otherwise
13 if  $\mathbf{P}$  is not a permutation matrix then
14   return  $\perp$ ;
15 return  $\mathbf{P}$ 

```

so that $\mathbf{P}^\top \mathbf{X} \mathbf{P} = \mathbf{Y}$. This is done by matching rows according to the multiset formed by their entries: if for row j of $\mathbf{Y}$ there is a unique row in $\mathbf{X}$, say, row i , having the same entry multiset, then the permutation must send i to j . This matching among rows is performed for all rows of $\mathbf{Y}$: if it is successful for every row, then the algorithm found the permutation. If, for a row in $\mathbf{Y}$, there is more than one matching rows in $\mathbf{X}$, then the algorithm returns a failure.

Observe that there exist more precise algorithms to solve GIP. Yet, as we motivate next, this algorithm is expected to succeed with large probability when both q and n are large enough.

We first recall a result which will be instrumental in our analysis.

Lemma 2 [RS09] Let $(\mathbf{a}, \mathbf{b}) \xleftarrow{\$} \mathbb{F}_q^n \times \mathbb{F}_q^n$. If $q = 2$, then $\Pr[\text{Count}(\mathbf{a}) = \text{Count}(\mathbf{b})] = \binom{2n}{n} 2^{-n}$ and, for growing n , $\Pr[\text{Count}(\mathbf{a}) = \text{Count}(\mathbf{b})] \sim f_{2,n} = \frac{1}{\sqrt{\pi n}}$.

If $q \geq 3$, then $\Pr[\text{Count}(\mathbf{a}) = \text{Count}(\mathbf{b})] = \binom{2n}{n} 2^{-n} \leq \frac{1}{q^n} \cdot f_{q,n}$, where

$$f_{q,n} = \begin{cases} n! & \text{if } q \geq n, \\ \frac{n!}{(v!)^{q(v+1)-n} ((v+1)!)^{n-qv}} & \text{if } q < n, \end{cases}$$

with $v = \lfloor n/q \rfloor$. For constant q and growing n , then $\Pr[\text{Count}(\mathbf{a}) = \text{Count}(\mathbf{b})]$ is upper bounded by $\frac{q^{\frac{q}{2}}}{(4\pi n)^{\frac{q}{2}-\frac{1}{2}}}$.

Remark 6 For $q = 2$ we have been able to derive an exact formula as a collision in the computation of **Count** means that the vectors $\mathbf{a}$ and $\mathbf{b}$ have the same Hamming weight. For $q \geq 3$, having the same Hamming weight is a necessary but not sufficient condition and one has to consider also how the number of non-null entries distribute: this makes computations rather cumbersome, especially if one wants to derive a closed form formula. This is the reason why we have relied for a much easier-to-handle upper bound.

Remark 7 For constant q , the quantity $\frac{f_{q,n}}{q^n}$ is in $O\left(n^{\frac{1-q}{2}}\right)$.

We now prove that, for any prime $q \geq 5$ and growing n , the probability that a random $n \times n$ matrix has two rows having the same entry multiset is in $O(1)$.

Lemma 3 Let $\mathbf{X} \xleftarrow{\$} \mathbb{F}_q^{n \times n}$; then, for constant $q \geq 5$ and growing n , the probability that all rows of $\mathbf{X}$ have distinct entries multiset is in $O(1)$. For $q < 5$, the probability is in $o(1)$.

Proof. We know that, for any row of $\mathbf{X}$, the number of vectors having the same entries multiset is not greater than $f_{q,n}$. Then, the probability we are searching for is at least

$$\prod_{i=1}^{n-1} 1 - \frac{i \cdot f_{q,n}}{q^n} > \left(1 - \frac{(n-1) \cdot f_{q,n}}{q^n}\right)^{n-1} > \left(1 - \frac{n \cdot f_{q,n}}{q^n}\right)^n.$$

We now bound $\frac{f_{q,n}}{q^n}$ with $\frac{q^{q/2}(4\pi)^{q/2-1/2}}{n^{q/2-1/2}}$ and get

$$\begin{aligned} \left(1 - \frac{n \cdot f_{q,n}}{q^n}\right)^n &> \left(1 - \frac{n \cdot q^{q/2}(4\pi)^{q/2-1/2}}{n^{q/2-1/2}}\right)^n \\ &= \left(1 - \frac{q^{q/2}(4\pi)^{q/2-1/2}}{n^{q/2-3/2}}\right)^n \\ &\sim e^{-(4\pi)^{(1-q)/2} \cdot q^{\frac{q}{2}} \cdot n^{(5-q)/2}}. \end{aligned}$$

For $q = 5$, the exponent is a constant in n . For any $q \geq 7$, the exponent grows as the product between a negative constant, given by $-(4\pi)^{(1-q)/2} \cdot q^{\frac{q}{2}}$, and a polynomial in n that decays as fast as $1/\sqrt{n}$. Hence, for growing n , the exponent is in $o(1)$ and the above bound approaches 1. $\square$

We can now use the above Lemma to estimate the probability that, for a random adjacency matrix $\mathbf{X} \in \mathbb{F}_q^{n \times n}$, all rows have distinct entries multisets: when this condition is met, Algorithm 2 can successfully solve search GIP. Observe that this requires to heuristically assume that the symmetry of the adjacency matrix does not influence the probability. We have verified this heuristic with

numerical simulations on random symmetric matrices, for several values of q and growing n ; for each parameter set, we have measured the probability that at least two rows have the same entries multiset. The results are shown in Figure 9. As we can see, the figure confirms the heuristic: for $q = 2$ or $q = 3$, the probability decreases as n increases, for $q = 5$ the probability stays approximately constant while, for $q \geq 7$, it approaches 1.

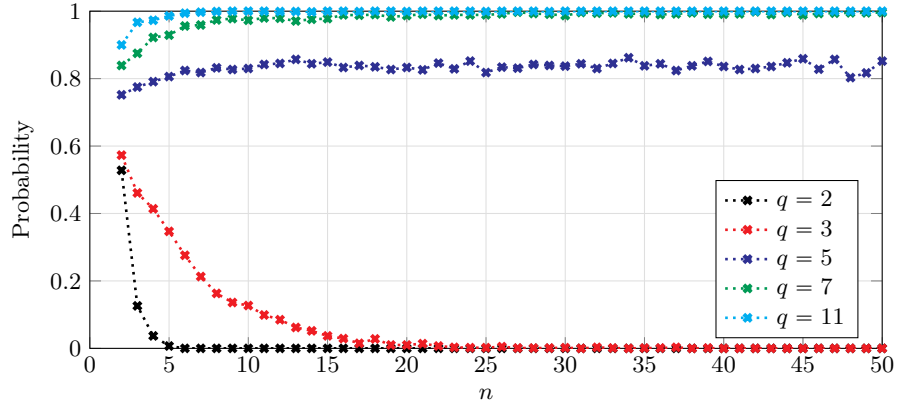


Fig. 9: Simulated probability of having at least two rows with the same multiset entries, for random $n \times n$ symmetric matrices over $\mathbb{F}_q$. For each pair (q, n) , we have generated 1000 matrices.